

zkTransformer: Scalable Zero-Knowledge Proofs for LLM Inference via Einsum Arithmetization and Structured Exponentiation Lookups

Bing-Jyue Chen
UIUC
USA
bjchen4@illinois.edu

Lilia Tang
UIUC
USA
liliat2@illinois.edu

Daniel Kang
UIUC
USA
ddkang@illinois.edu

Abstract

Large language models (LLMs) are widely deployed through cloud APIs; however, their inference processes are opaque and cannot be externally audited without revealing proprietary model parameters. Zero-knowledge proofs (ZKPs) enable verifiable inference while preserving model confidentiality, but existing zero-knowledge machine learning (zkML) systems remain difficult to deploy in practice due to limitations in developer usability, completeness of prover implementations, and the cost of lookup operations.

We present zkTransformer, an end-to-end zkML framework for transformer inference that addresses these challenges while achieving high prover performance. To improve usability and correctness, we introduce Einsumcheck, a new sumcheck-based arithmetization that lifts linear tensor operations to an AI-friendly abstraction based on Einstein summation (Einsum), enabling automatic compilation of such operations into a *single* Sumcheck instance without manual circuit engineering over low-level addition and multiplication gates. To reduce the cost of nonlinear operations, we further propose StructuredExp, a structured lookup technique for exponentiation that eliminates the need for table-sized commitments.

Together, these techniques provide a unified and model-agnostic framework for verifiable transformer inference, supporting standard architectures such as GPT-2 and LLaMA2-7B. We implement and open-source zkTransformer. On GPT-2, it achieves a prover time of 3.4 seconds per token, representing a 5–10× speedup over state-of-the-art systems such as zkLLM and zkGPT.

CCS Concepts

- **Security and privacy** → **Logic and verification**; *Cryptography*;
- **Computing methodologies** → *Natural language processing*.

Keywords

zero-knowledge proofs, machine learning, transformers, large language models, verifiable computation

ACM Reference Format:

Bing-Jyue Chen, Lilia Tang, and Daniel Kang. 2025. zkTransformer: Scalable Zero-Knowledge Proofs for LLM Inference via Einsum Arithmetization and Structured Exponentiation Lookups. In *Proceedings of ACM Conference (Conference'25)*. ACM, New York, NY, USA, 20 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large language models (LLMs) are now widely deployed through cloud APIs to support applications such as chat assistants [32], search [34], and code generation [3]. However, the inference is typically performed by the service provider and is opaque to external parties, requiring users and regulators to trust that the advertised model and inference procedure are executed faithfully. This concern is particularly salient for code generation, where incorrect or manipulated outputs may introduce security vulnerabilities [35].

Traditional auditing approaches that rely on direct access to model weights [6] are infeasible for modern LLM services, since the trained model parameters are typically proprietary and cannot be disclosed for verification without violating intellectual property constraints. Zero-knowledge proofs (ZKPs) [48] offer a solution by enabling a service provider to prove that an output was generated by executing a specified model architecture on a given input, without revealing the model parameters. This enables verifiable inference while preserving the confidentiality of model weights.

Recent zero-knowledge machine learning (zkML) systems [9, 10, 28, 36, 46, 58] have demonstrated the feasibility of verifiable inference for LLMs. Among them, Sumcheck-based protocols [36, 46] (e.g., zkGPT [36] based on GKR [15]) achieve superior prover performance compared to alternative approaches. For example, Plonk-based zkML systems [10] require 1.1 hours to prove GPT-2 inference, whereas zkGPT completes the same task in 22 seconds.

Despite this progress, existing Sumcheck-based zkML systems for LLM inference still present several challenges for practical deployment, including (1) developer usability, (2) completeness of the prover implementations, and (3) the cost of lookup operations.

To address these challenges, we present zkTransformer, a zero-knowledge machine learning (zkML) framework that achieves a 5–10× prover speedup over state-of-the-art systems zkLLM [46] and zkGPT [36] on GPT-2. We elaborate on these three directions and compare prior work with zkTransformer below.

First, both zkLLM [46] and zkGPT [36] require developers to manually express model computations as arithmetic circuits over low-level addition and multiplication gates. As a result, even minor model changes, such as modifying the sequence length or replacing an operator, require developers to rewire the low-level

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'25, City, Country

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2025/XX
<https://doi.org/XXXXXXX.XXXXXXX>

circuit. We identify a key reason why these Sumcheck-based frameworks [36, 46] struggle to support AI-friendly user interfaces: expressing general linear tensor operations within current GKR-based arithmetization schemes [15] requires substantial systems engineering effort. In practice, implementing common linear operations in LLMs often involves composing multiple Sumcheck instances with the low-level addition and multiplication gates. This process demands a deep understanding of the underlying arithmetization, thereby limiting the range of supported models and hindering broader adoption within the AI community.

Moreover, manual low-level arithmetization can introduce subtle issues that affect the completeness of the protocol even when the developers are familiar with cryptographic techniques. Indeed, representative works on verifiable LLM inference, zkGPT [36] and zkLLM [46], exhibit instances where the prover completeness is affected (i.e., even an honest prover may fail to convince the verifier). For example, the batched matrix multiplication in zkGPT overlooks the fact that the batch index appears in both operands, which breaks multilinearity and thus affects completeness.¹ A similar multilinearity issue arises in the implementation of element-wise multiplication in zkLLM [46].²

To address both usability and completeness, we introduce a new Sumcheck arithmetization, EinSumcheck, which enables practitioners to express LLM layers using a unified and AI-friendly abstraction: Einstein summation (Einsum) [11, 39]. Unlike standard GKR arithmetizations, which often require multiple Sumcheck instances for various linear operations, EinSumcheck compiles any linear operation expressible in Einsum notation into a *single* Sumcheck instance. Furthermore, traditional GKR-based approaches often require commitments to wiring polynomials (i.e., auxiliary polynomials that enforce data consistency among intermediate wires) to enable efficient verifier evaluation during the protocol, incurring additional overhead in both commitments and evaluation proofs. In contrast, EinSumcheck avoids such commitments beyond the witness polynomials because its wiring polynomials possess a special structure that allows the verifier to evaluate them efficiently on its own. Thus, EinSumcheck retains *succinct* verifier time and proof size, both growing sublinearly in the circuit size, without requiring prover assistance for evaluating the wiring polynomials.

The final challenge lies in the prover overhead of lookup protocols, where existing zkML systems incur substantial costs from committing to large lookup tables and auxiliary polynomials during range and exponentiation checks [29, 36, 46]. Upon examining the implementations of zkGPT and zkLLM, we observe that both use LogUp-style lookup protocols [17]³, which require the prover to commit to the lookup table as well as auxiliary polynomials proportional to its size. Recent work on Sumcheck-based lookup arguments [41, 42] introduces *structured tables*, eliminating table-sized commitments and significantly reducing prover overhead; these techniques have already been adopted in other domains such as zkVMs [55]. The key idea is that when table entries follow a

specific structure, the verifier can compute evaluations of the corresponding table polynomial independently, removing the need for prover-side commitments.

In zkTransformer, we leverage structured tables for both range and exponentiation checks to reduce the lookup cost. The main challenge is that while range tables are naturally structured [41], exponentiation tables have traditionally been considered unstructured [36]. As a result, existing zkML systems incur substantial overhead when proving exponentiation, prevalent in operations such as softmax layers. We address this gap by introducing StructuredExp, a technique that reformulates exponentiation relations to admit structured lookup representations, thereby eliminating the need for large table-sized commitments.

We integrate these techniques into an end-to-end framework that exposes AI-friendly LLM operators (e.g., einsum and softmax) and automatically compiles them into efficient GKR-based proof circuits. This design removes the need for manual low-level circuit engineering, allowing practitioners to express transformer inference using familiar abstractions while ensuring prover completeness. To enable efficient end-to-end proof generation, we combine EinSumcheck and StructuredExp with existing optimizations such as batched verification [53] and fast polynomial evaluation [36]. The resulting prover requires only 3.4 seconds per token to prove GPT-2 inference, achieving a 5–10× speedup over state-of-the-art systems zkLLM [46] and zkGPT [36]. We open-source our implementation [2]. Our contributions are summarized as follows.

- (1) We propose the EinSumcheck compiler, a new Sumcheck arithmetization that lifts linear tensor operations to the Einsum abstraction, eliminating the need for manual circuit decomposition in prior work and improving both developer usability and prover completeness.
- (2) We introduce StructuredExp, which arithmetizes exponentiation relations using only structured lookup tables, eliminating the need for table-sized commitments. This enables efficient proof generation of nonlinearities using range and exponentiation checks within modern structured lookup protocols [41].
- (3) We implement zkTransformer, incorporate existing optimization with our techniques, and evaluate it on standard transformer models, demonstrating both high prover efficiency and strong usability. We release our implementation [2] to support reproducibility and future research.

2 Preliminaries

In this section, we introduce the notation and background necessary for understanding zkTransformer. We begin with Einstein summation, the core abstraction underlying our usability design (Sec. 2.1). Next, we review the components in LLMs (Sec. 2.2) and the quantization techniques used in zkML (Sec. 2.3). Finally, we provide an overview of the cryptographic primitives that zkTransformer builds upon, including the GKR protocol (Sec. 2.4) and Shout lookup (Sec. 2.5). We also review the conditions for the prover completeness to hold in Lemma 2.2.

2.1 Einsum Notation

Einstein summation (Einsum) is a widely adopted language in the AI community for expressing linear tensor operations [39]. To improve

¹See Eq. (8) in zkGPT [36]. Please also see Lemma 2.2 and our example in Sec. 3.

²See the released script github.com/jvhs0706/zkllm-ccs2024/blob/993311/proof.cu

³While zkGPT also mentions Lasso-style lookups [42] in the paper, their released script implements a LogUp-style one.

the usability of zkTransformer, we design a new arithmetization for various Einsum-expressible equations. We briefly review the Einsum notation below.

Einsum [11] expresses linear tensor computations by explicitly specifying index relationships among operands. An Einsum expression is written as

$$I_1, \dots, I_m \rightarrow O,$$

where each input tensor $T^{(r)}$ is annotated with an ordered tuple of indices I_r , and the output tensor is annotated with indices O .

Indices are categorized into two types. *Free indices* appear in both the output O and some inputs, and determine the output shape. *Summation indices* appear only in inputs and are reduced over. Let $\mathcal{F}$ and $\mathcal{S}$ denote the sets of free and summation indices, respectively. For any assignment $\mathbf{f} \in \prod_{i \in \mathcal{F}} \mathcal{D}_i$, the output tensor Z is defined as

$$Z[\mathbf{f}] = \sum_{\mathbf{s} \in \prod_{j \in \mathcal{S}} \mathcal{D}_j} \prod_{r=1}^m T^{(r)}[\pi_r(\mathbf{f}, \mathbf{s})],$$

where π_r selects the index tuple for operand $T^{(r)}$ according to its annotation I_r .

Example (Batched Matrix Multiplication). The Einsum expression $\text{bij}, \text{bjk} \rightarrow \text{bik}$ with $X \in \mathbb{R}^{B \times I \times J}$ and $Y \in \mathbb{R}^{B \times J \times K}$ has free indices $\{b, i, k\}$ and summation index $\{j\}$, yielding

$$Z[b, i, k] = \sum_{j=1}^J X[b, i, j] \cdot Y[b, j, k].$$

2.2 Transformer

zkTransformer is a new framework for verifiable inference of LLMs. Modern LLMs consist of a token embedding layer and a sequence of transformer blocks [51], each of which contains linear layers (e.g., matrix multiplications), normalization layers (e.g., RMSNorm), element-wise nonlinearities (e.g., GeLU [22] or SiLU [12]), and residual connections [21].

A key observation is that almost all linear operations in LLMs can be expressed using Einsum notation [39], including matrix multiplication, element-wise multiplication (e.g., $i, i \rightarrow i$), and summation (e.g., $i, j \rightarrow i$). Moreover, prior zkML work shows that common nonlinearities in LLMs (including Softmax, Sigmoid, SiLU [12], and GeLU [22]) can be reduced to combinations of range and exponentiation checks using lookup protocols [29, 36, 46]. For instance, we can prove the Softmax operation for a vector t with an exponentiation check as follows:

$$\frac{e^{t_i}}{\sum_i e^{t_i}} = \frac{e^{t_i - \max_i(t_i)}}{\sum_i e^{t_i - \max_i(t_i)}} = e^{t_i - \max_i(t_i) - r}, \quad r := \ln \left(\sum_i e^{t_i - \max_i(t_i)} \right). \quad (1)$$

where t_i denotes the i -th element of the vector. Importantly, there is no need to prove the correctness of $\max(\cdot)$ or $\ln(\cdot)$ explicitly. Instead, the prover can commit to $r' = \max_i(t_i) + r$, perform element-wise subtraction, apply an exponentiation check, and verify that the resulting values sum to approximately 1. Another example is SiLU used in LLaMA [50], defined as $\text{SiLU}(x) = x \cdot \text{Sigmoid}(x)$. The Sigmoid function [19] is

$$\frac{1}{1 + e^{-x}} = \frac{e^x}{e^0 + e^x},$$

which can be computed using Softmax. Similarly, prior work [36] shows how to arithmetize GeLU in GPT-style models using only range and exponentiation checks. We adopt this abstraction and focus on improving the efficiency of proving these two classes of lookup constraints (i.e., range and exponentiation).

2.3 Quantization in zkML

ZKP systems operate over prime fields, so zkML frameworks quantize real values into integers [9, 10, 28, 36, 46]. We briefly review the general quantization framework used in prior work; zkTransformer's specific quantization choices are described in Sec. 6.

In standard fixed-point quantization, a real value x is represented as

$$x = S(q - z),$$

where S is a scale factor and z is a zero-point. Setting $z = 0$ gives $q = \text{round}(x/S)$. Addition with equal scales reduces to integer addition, while multiplication requires rescaling. Given $x = S_x q_x$ and $y = S_y q_y$, we compute $z = S_z q_z$ via

$$q_z = \text{round} \left(\frac{C_1}{C_2} \cdot q_x q_y \right), \quad \frac{C_1}{C_2} \approx \frac{S_x S_y}{S_z},$$

where C_1, C_2 are public integers. This enables approximate real arithmetic using only field operations. Prior systems differ in how they choose scale factors: zkGPT [36] uses adaptive per-layer scales ($c, 2^m$) determined by a search algorithm based on weight magnitudes, while zkLLM [46] uses per-operation hardcoded scales (e.g., 2^{20} for gate projection, 2^{16} for other projections).

2.4 GKR Protocol for ML Inference

zkTransformer translates a quantized LLM computation into algebraic constraints and proves the constraints in ZKPs. The induced relations fall into two categories: *arithmetic relations* (i.e., linear operations), which are proven via the GKR protocol [56], and *lookup relations* (e.g., range and exponentiation checks), which are handled by Shout [41] (Sec. 2.5).

We defer full cryptographic details to Sec. D. The classic GKR protocol [15] is an interactive proof for layered arithmetic circuits of addition and multiplication gates, where each gate depends on at most two wires from the previous layer. To allow inputs from arbitrary earlier layers (e.g., model weights), prior work [56] introduced a GKR extension known as *ML-friendly GKR circuits* [28, 36], which allows static parameters to be accessed at arbitrary depths without explicit propagation.

At a high level, GKR applies the Sumcheck protocol [30] to recursively reduce a claim about the output layer to evaluation claims about earlier layers, eventually reaching the input layer. Its logarithmic interaction and quasi-linear prover complexity make it a common tool for verifiable computation.

We now introduce the notation used by GKR: multilinear extensions, Sumcheck and the GKR arithmetization.

2.4.1 Multilinear Extensions. Multilinear extensions are the data carriers in a Sumcheck-based protocol [15, 41, 47, 56].

Definition 2.1 (Multilinear polynomials). A polynomial $g : \mathbb{F}^\mu \rightarrow \mathbb{F}$ is *multilinear* if it has degree at most one in each variable.

LEMMA 2.2 (MULTILINEAR EXTENSIONS (ADAPTED FROM [41])). *For any function $f : \{0, 1\}^\mu \rightarrow \mathbb{F}$, there exists a unique multilinear polynomial $\tilde{f} : \mathbb{F}^\mu \rightarrow \mathbb{F}$ such that $\tilde{f}(x) = f(x)$ for all $x \in \{0, 1\}^\mu$.*

Lemma 2.2 implies if (1) two polynomials agree on $\{0, 1\}^\mu$, and (2) they are both *multilinear*, then they are identical. As we mentioned in the introduction (also Sec. 3), even experienced researchers may overlook the second condition and break the completeness.

In zkTransformer, we use two special extensions below.

Lagrange basis polynomial $\tilde{\text{eq}}$. Let $\text{eq} : \{0, 1\}^\mu \times \{0, 1\}^\mu \rightarrow \mathbb{F}$ be $\text{eq}(x, y) = 1[x = y]$. Its multilinear extension is

$$\tilde{\text{eq}}(x, y) = \prod_{i=1}^{\mu} (x_i y_i + (1 - x_i)(1 - y_i)). \quad (2)$$

It can be evaluated in $O(\mu)$ time at any $(r, r') \in \mathbb{F}^\mu \times \mathbb{F}^\mu$. Following prior work [41, 42], we call any $O(\log n)$ -variate polynomial evaluable in $O(\log n)$ time *structured*; thus $\tilde{\text{eq}}$ is structured. For any $f : \{0, 1\}^\mu \rightarrow \mathbb{F}$ and its multilinear extension $\tilde{f}$, the following holds

$$\tilde{f}(y) = \sum_{b \in \{0, 1\}^\mu} f(b) \cdot \tilde{\text{eq}}(b, y),$$

so $\tilde{\text{eq}}$ is also known as the Lagrange basis polynomial [26].

Multilinear extensions for tensors. We model an n -dimensional quantized tensor T of shape $(D_1, \dots, D_n)$ as a function

$$T : [D_1] \times \dots \times [D_n] \rightarrow \mathbb{F}, \quad [D] := \{0, 1, \dots, D - 1\},$$

and write $T \in \mathbb{F}^{D_1 \times \dots \times D_n}$. We assume each $D_k = 2^{d_k}$ (padding with zeros if needed) and encode indices in binary to view T as a function over the Boolean hypercube:

$$\tilde{T} : \{0, 1\}^{d_1} \times \dots \times \{0, 1\}^{d_n} = \{0, 1\}^d \rightarrow \mathbb{F}, \quad d = \sum_{k=1}^n d_k,$$

where $\tilde{T}$ is the unique multilinear polynomial that agrees with T on Boolean inputs. Concretely,

$$\tilde{T}(i_{1,1}, \dots, i_{1,d_1}, \dots, i_{n,1}, \dots, i_{n,d_n}) = T[i_1, \dots, i_n]$$

given $i_k = \sum_{j=1}^{d_k} i_{k,j} 2^{j-1}$, $i_{k,j} \in \{0, 1\}$. We freely switch between an index $i \in [0, n-1]$ and its binary representation in $i \in \{0, 1\}^{\log n}$ (starting from the LSB), and we write $\tilde{T}(i_1, \dots, i_n)$ when the bit-level expansion is clear. This representation allows Sumcheck-based protocols to verify tensor computations via polynomial identities.

2.4.2 Sumcheck and GKR. The Sumcheck protocol [30] is a core subroutine in many protocols (e.g., [7, 41, 56]). Given a μ -variate polynomial f over $\mathbb{F}$, Sumcheck proves the claimed sum

$$H := \sum_{x \in \{0, 1\}^\mu} f(x). \quad (3)$$

Sumcheck runs for μ rounds and reduces (3) to checking the claimed evaluation $f(r)$ at a random point $r \in \mathbb{F}^\mu$ (see Protocol E).

GKR applies Sumcheck to layered circuits. For an L -layer (ML-friendly) GKR circuit, let $\tilde{V}_i$ denote the multilinear extension of the value vector at layer i , with $\tilde{V}_0$ the input layer. The prover commits to $\tilde{V}_0$ and claims $\tilde{V}_L(r_L)$ at a verifier-chosen r_L . Each Sumcheck instance reduces an output-layer claim to evaluation claims about earlier layers, iterating until reaching $\tilde{V}_0$, after which the prover opens the required evaluations under the polynomial commitments.

For completeness, the classic GKR arithmetization [15] expresses each layer polynomial as

$$\begin{aligned} \tilde{V}_i(z) = & \sum_{x, y \in \{0, 1\}^s} \widetilde{\text{add}}_{i-1}(z, x, y) (\tilde{V}_{i-1}(x) + \tilde{V}_{i-1}(y)) \\ & + \sum_{x, y \in \{0, 1\}^s} \widetilde{\text{mult}}_{i-1}(z, x, y) \tilde{V}_{i-1}(x) \tilde{V}_{i-1}(y), \end{aligned} \quad (4)$$

where each gate at layer i is either an addition or multiplication gate wired to a unique pair (x, y) in layer $i-1$. The polynomials $\widetilde{\text{add}}_{i-1}$ and $\widetilde{\text{mult}}_{i-1}$ are *wiring predicates* that equal 1 on Boolean inputs exactly for valid parent-child wiring of the corresponding gate type, and 0 otherwise. The ML-friendly extension [56] generalizes this arithmetization to allow wiring from arbitrary earlier layers, but the role of wiring predicates remains the same.

2.5 Shout Lookup Protocol

Shout [41] is the state-of-the-art Sumcheck-based lookup, which zkTransformer adopts to handle nonlinearities in LLMs. Given a length- m vector V and a length- n table T (with $n \gg m$),⁴ to prove that all elements of V lie in T , one can construct a Boolean matrix M such that each row of M contains exactly one entry equal to 1, and prove that $V = MT$. By forming the multilinear extensions $\tilde{V}$, $\tilde{M}$, and $\tilde{T}$, this relation can be proven using the Sumcheck-based matrix multiplication protocol [47] for the following equality, where r is a challenge sampled by the verifier:

$$\tilde{V}(r) = \sum_{y \in \{0, 1\}^{\log n}} \tilde{M}(r, y) \tilde{T}(y). \quad (5)$$

When the table $\tilde{T}(y)$ is structured, Shout [41] proposes several techniques to accelerate the above Sumcheck prover with a sparse $\tilde{M}(x, y)$, reducing the prover time to nearly $O(m)$ instead of $O(n)$. In addition, Shout includes auxiliary Sumcheck instances to prove that $\tilde{M}(x, y)$ is indeed the multilinear extension of a Boolean matrix M in which each row contains exactly one 1.

Another important property of Shout is that committing to $\tilde{M}(x, y)$ is inexpensive when using elliptic-curve-based polynomial commitment schemes such as [24, 26], due to the sparsity of M . Once the Sumcheck protocol reduces the relation to evaluation claims of $\tilde{M}$ and $\tilde{T}$ at random points, the prover can efficiently provide evaluation proofs for $\tilde{M}$. With modern polynomial commitment schemes such as KZH [24], the prover time is sublinear in the polynomial size. Moreover, the prover does not need to provide evaluation proofs for the table polynomial $\tilde{T}$, because it is public and structured; the verifier can evaluate it directly in $O(\log n)$ time.

3 EinSumcheck: An Arithmetization for Linear Operations in GKR

In this section, we introduce a new arithmetization for the GKR protocol, which serves as the main building block for proving linear operations in zkTransformer.

⁴This assumption holds in zkML: for range checks, $n = 2^{24}$ while $m = 2^{12}$ at GPT-2 scale; for exponentiation, zkGPT's experiments use table size $n \approx 655K$ vs. lookups $m \approx 67K$. More importantly, when using structured tables as in StructuredExp, the prover cost does not linearly depend on n , because the verifier evaluates the structured table polynomial directly without requiring a commitment and its evaluation proof.

We highlight two main reasons why EinSumcheck benefits zkML. First, it improves usability. Existing GKR-based approaches require manually encoding linear operations using low-level addition and multiplication constraints since the classic arithmetization (Eq. (4)) only supports these gates within a Sumcheck instance. This makes it cumbersome to express common linear operations in LLM inference. In contrast, EinSumcheck provides a more natural abstraction by allowing practitioners to directly translate an Einsum equation into a single Sumcheck instance, eliminating the need for manual constraint construction.

Second, it helps ensure the completeness of the prover implementation. Automatic compilation of any Einsum-expressible operation into a single Sumcheck without manual arithmetization can help eliminate the prover completeness issues that even experienced developers may make (see the below example).

Additionally, EinSumcheck retains a *succinct* verifier without requiring commitments to any wiring predicates, since all such predicates are *structured*. In contrast, the wiring predicates (i.e., polynomials `add` and `mult`) in Eq. (4) are not necessarily *structured*. Consequently, either the prover must commit to these predicates and provide evaluation proofs for them, or the verifier must evaluate them directly, increasing verifier overhead.⁵

We now describe how to arithmetize a general Einsum expression into polynomial relations suitable for verification using the Sumcheck protocol. We will start with an example to give readers better intuition about the EinSumcheck compiler. After the example, we introduce the formal arithmetization and its security proof. We also provide the corresponding prover algorithm in Sec. F.

Example (Batched Matrix Multiplication). Recall that the Einsum expression `bij, bjk → bik` means

$$Z[b, i, k] = \sum_{j=0}^{J-1} X[b, i, j] \cdot Y[b, j, k]$$

for all $b \in [0, B-1]$, $i \in [0, I-1]$, $k \in [0, K-1]$. Given their multilinear extensions $\tilde{Z}$, $\tilde{X}$, and $\tilde{Y}$, the following holds for all $b \in \{0, 1\}^{\log B}$, $i \in \{0, 1\}^{\log I}$, $k \in \{0, 1\}^{\log K}$, by the definition of batched matrix multiplication and multilinear extensions for tensors:

$$\tilde{Z}(b, i, k) = \sum_{j \in \{0, 1\}^{\log J}} \tilde{X}(b, i, j) \cdot \tilde{Y}(b, j, k). \quad (6)$$

Equation (8) in zkGPT [36] suggests that one can directly use the Sumcheck protocol to prove (6) by checking, for a verifier challenge $r = (r_b, r_i, r_k) \in \mathbb{F}^{\log B + \log I + \log K}$, whether

$$\tilde{Z}(r_b, r_i, r_k) = \sum_{j \in \{0, 1\}^{\log J}} \tilde{X}(r_b, r_i, j) \cdot \tilde{Y}(r_b, j, r_k)$$

holds. However, this formulation *violates completeness*: even an honest prover with a correct witness cannot convince the verifier.

To see why, observe that the left-hand side $\tilde{Z}(r_b, r_i, r_k)$ is multilinear, and hence has degree 1 in the variable r_b . By contrast, the right-hand side contains the product $\tilde{X}(r_b, r_i, j) \cdot \tilde{Y}(r_b, j, r_k)$, where r_b appears in both factors, making the product at least degree 2 in r_b . While both sides agree on Boolean inputs (i.e., when $r_b \in \{0, 1\}$),

⁵We refer readers to the open-source implementations of zkCNN [28] and zkGPT [36], where wiring predicates are not committed and must instead be evaluated directly by the verifier.

they are *distinct polynomials* over the field $\mathbb{F}$ by Lemma 2.2. Consequently, when the verifier samples a random challenge $r_b \in \mathbb{F}$, the two sides will disagree with high probability, causing verification to fail even when the underlying computation is correct.

Instead, in this work, we leverage the Lagrange basis for multilinear polynomials, denoted by $\tilde{eq}$, to enforce multilinearity. Specifically, Eq. (6) can be rewritten as

$$\tilde{Z}(b, i, k) = \sum_{\substack{b' \in \{0, 1\}^{\log B} \\ j \in \{0, 1\}^{\log J}}} \tilde{eq}(b, b') \cdot \tilde{X}(b', i, j) \cdot \tilde{Y}(b', j, k). \quad (7)$$

It is clear that both sides agree on all Boolean inputs $b \in \{0, 1\}^{\log B}$, $i \in \{0, 1\}^{\log I}$, and $k \in \{0, 1\}^{\log K}$. Moreover, both sides are multilinear polynomials in variables b, i, k . By Lemma 2.2, the two polynomials must be identical, and hence they agree over all field points $b \in \mathbb{F}^{\log B}$, $i \in \mathbb{F}^{\log I}$, and $k \in \mathbb{F}^{\log K}$. Consequently, one can safely use the Sumcheck protocol to prove that

$$\tilde{Z}(r_b, r_i, r_k) = \sum_{\substack{b' \in \{0, 1\}^{\log B} \\ j \in \{0, 1\}^{\log J}}} \tilde{eq}(r_b, b') \cdot \tilde{X}(b', r_i, j) \cdot \tilde{Y}(b', j, r_k).$$

The example illustrates the construction used in EinSumcheck, which follows directly from the semantics of Einsum. Summation indices must be explicitly summed over (e.g., j in Eq. (6)), as in standard tensor contraction. However, free indices that appear in multiple input operands would otherwise increase the degree of the product polynomial. To preserve multilinearity, we insert the Lagrange basis $\tilde{eq}$ and sum over the Boolean hypercube for such indices. For free indices that appear in only one input operand, they can be safely substituted by verifier randomness.

When $B = 1$ (i.e., $\log B = 0$), Eq. (7) reduces to the standard matrix multiplication arithmetization used in the MatMul protocol [47] without the Lagrange basis. Concretely, EinSumcheck generalizes MatMul [47] along three axes: (a) arbitrary Einsum contractions, (b) multi-operand contractions in a single Sumcheck, (c) operands with different variable counts in linear prover time. Beyond these, the framework-level contribution is automatic compilation: any Einsum-expressible operation becomes a single Sumcheck without manual arithmetization.

We now formally introduce the general arithmetization.

Categories of Einsum indices. Consider an Einsum expression $I_1, \dots, I_m \rightarrow O$ with m input tensors $T^{(1)}, \dots, T^{(m)}$ and their multilinear extensions $\tilde{T}^{(1)}, \dots, \tilde{T}^{(m)}$, and output tensor Z and its extension $\tilde{Z}$. Let $\mathcal{F}$ denote the set of free indices (those appearing in O), and $\mathcal{S}$ denote the set of summation indices (those appearing in the inputs but not in O).

We further partition the free indices into two subsets:

- **Degree-1 free indices** $\mathcal{F}_1$: indices that appear in exactly one input tensor annotation.
- **High-degree free indices** $\mathcal{F}_{\geq 2}$: indices that appear in two or more input tensor annotations.

Thus, $\mathcal{F} = \mathcal{F}_1 \cup \mathcal{F}_{\geq 2}$, and the index sets $\mathcal{S}, \mathcal{F}_1, \mathcal{F}_{\geq 2}$ form a disjoint partition of all indices.

The core compiler. Let $r = (r_u)_{u \in \mathcal{F}}$ be a random evaluation point chosen by the verifier as the challenge. The prover claims the value

$\tilde{Z}(r)$. We express this value as a sum over summation indices and high-degree free indices.

THEOREM 3.1 (EINSUM ARITHMETIZATION). *Let $\tilde{Z}$ be defined by an Einsum contraction of multilinear input polynomials $\tilde{T}^{(1)}, \dots, \tilde{T}^{(m)}$, where each operand $\tilde{T}^{(t)}$ is indexed by an ordered index tuple I_t . Let $\mathcal{S}$ be the set of summation indices, and let the free indices be partitioned into degree-1 free indices $\mathcal{F}_1$ and high-degree free indices $\mathcal{F}_{\geq 2}$. Let $r = (r_u)_{u \in \mathcal{F}}$ be a verifier-chosen evaluation point.*

Then the output value $\tilde{Z}(r)$ equals

$$\sum_{s \in \{0,1\}^{|\mathcal{S}|}} \sum_{\mathbf{h} \in \{0,1\}^{|\mathcal{F}_{\geq 2}|}} \left(\prod_{u \in \mathcal{F}_{\geq 2}} \tilde{eq}(h_u, r_u) \right) \cdot \prod_{t=1}^m \tilde{T}^{(t)}(\pi_t(\mathbf{h}, \mathbf{s}, \mathbf{r}_{\mathcal{F}_1})),$$

where:

- $\mathbf{s}$ assigns values to summation indices in $\mathcal{S}$;
- $\mathbf{h}$ assigns values to high-degree free indices in $\mathcal{F}_{\geq 2}$;
- indices in $\mathcal{F}_1$ are substituted directly by verifier randomness $\mathbf{r}_{\mathcal{F}_1}$;
- π_t maps the combined assignment $(\mathbf{h}, \mathbf{s}, \mathbf{r}_{\mathcal{F}_1})$ to the ordered index tuple required by operand $\tilde{T}^{(t)}$ according to its annotation I_t .

Moreover, the Sumcheck protocol can be used to reduce the evaluation claim of $\tilde{Z}(r)$ to evaluation claims of the input polynomials $\tilde{T}^{(t)}$ and the Lagrange basis polynomials $\tilde{eq}$.

PROOF. The left-hand side $\tilde{Z}(r)$ is multilinear in the coordinates of r by definition of the multilinear extension of the output tensor. The right-hand side is also multilinear in r : each factor $\tilde{eq}(h_u, r_u)$ is multilinear in r_u for $u \in \mathcal{F}_{\geq 2}$, and for $u \in \mathcal{F}_1$ the randomness r_u is substituted into exactly one operand $\tilde{T}^{(t)}$, which preserves multilinearity since each operand is multilinear in every coordinate. Hence both sides are multilinear polynomials in the coordinates of r . By uniqueness of multilinear extension (Lemma 2.2), it suffices to verify the identity on all Boolean assignments $r = f \in \{0, 1\}^{|\mathcal{F}|}$.

Fix any $f \in \{0, 1\}^{|\mathcal{F}|}$. Evaluating the right-hand side at $r = f$, we have $\tilde{eq}(h_u, f_u) = \mathbf{1}[h_u = f_u]$ for Boolean points, so the sum over $\mathbf{h} \in \{0, 1\}^{|\mathcal{F}_{\geq 2}|}$ collapses to the single assignment $\mathbf{h} = f_{\mathcal{F}_{\geq 2}}$:

$$\begin{aligned} \sum_{\mathbf{h}} \left(\prod_{u \in \mathcal{F}_{\geq 2}} \tilde{eq}(h_u, f_u) \right) \prod_{t=1}^m \tilde{T}^{(t)}(\pi_t(\mathbf{h}, \mathbf{s}, \mathbf{f}_{\mathcal{F}_1})) \\ = \prod_{t=1}^m \tilde{T}^{(t)}(\pi_t(f_{\mathcal{F}_{\geq 2}}, \mathbf{s}, \mathbf{f}_{\mathcal{F}_1})). \end{aligned}$$

Thus the right-hand side at $r = f$ becomes

$$\sum_{s \in \{0,1\}^{|\mathcal{S}|}} \prod_{t=1}^m \tilde{T}^{(t)}(\pi_t(f_{\mathcal{F}_{\geq 2}}, \mathbf{s}, \mathbf{f}_{\mathcal{F}_1})).$$

But assembling operand inputs from $(f_{\mathcal{F}_{\geq 2}}, \mathbf{s}, \mathbf{f}_{\mathcal{F}_1})$ is equivalent to assembling them from $(\mathbf{s}, f)$, i.e., $\pi_t(f_{\mathcal{F}_{\geq 2}}, \mathbf{s}, \mathbf{f}_{\mathcal{F}_1}) = \pi_t(\mathbf{s}, f)$ for every t . Therefore the above equals

$$\sum_{s \in \{0,1\}^{|\mathcal{S}|}} \prod_{t=1}^m \tilde{T}^{(t)}(\pi_t(\mathbf{s}, f)),$$

which is exactly the definition of the Einsum contraction evaluated on Boolean indices, and hence equals $\tilde{Z}(f)$. Therefore, the two sides agree for all $f \in \{0, 1\}^{|\mathcal{F}|}$, and by uniqueness of multilinear

extension they agree for all $r \in \mathbb{F}^{|\mathcal{F}|}$, ensuring perfect completeness. $\square$

LEMMA 3.2 (SOUNDNESS OF EinSumcheck). *Suppose that each polynomial $\tilde{T}_i$, for $i \in [m]$, has at most v variables. Then the soundness error and knowledge soundness error of EinSumcheck are both at most $\frac{(m+1) \cdot v}{|\mathbb{F}|}$, where $|\mathbb{F}|$ denotes the size of the field over which Sumcheck is executed.*

PROOF. Each round of Sumcheck involves a univariate polynomial whose degree is at most $m + 1$. Since the maximum number of variables appearing in any $\tilde{T}_i$ is v , by Proposition 4.1 of [48], the total soundness error of the Sumcheck protocol is at most $\frac{(m+1) \cdot v}{|\mathbb{F}|}$. Furthermore, by Lemma 2.3 of [7], the knowledge soundness error of Sumcheck admits the same bound, completing the proof. $\square$

LEMMA 3.3 (SUCCINCTNESS WITHOUT COMMITMENT TO WIRING PREDICATES). *To ensure succinct verifier time and proof size, the EinSumcheck prover never needs to commit to any wiring predicates, unlike the standard GKR arithmetization.*

PROOF. At the end of the Sumcheck protocol for EinSumcheck, the verifier needs to check evaluation claims of $\tilde{eq}$ and $\tilde{T}^{(t)}$ for all $t \in [1, m]$. As in standard GKR, the prover may either provide evaluation proofs for $\tilde{T}^{(t)}$ or further reduce them to other claims using additional Sumcheck instances. The difference is that $\tilde{eq}$ (i.e., our wiring predicate) is structured and requires no evaluation proof: the verifier can evaluate it directly in sublinear time. $\square$

The remaining challenge is achieving a linear-time prover for the EinSumcheck arithmetization. Existing linear-time Sumcheck provers [47, 54] handle relations of the form $H = \sum_x \prod_t \tilde{T}^{(t)}(x)$, where each $\tilde{T}^{(t)}$ is a multilinear polynomial over the same variable set. By contrast, EinSumcheck induces Sumcheck relations with additional $\tilde{eq}$ factors and with operands that may depend on different sets of variables. At a high level, our prover reorganizes every two-operand EinSumcheck relation into a canonical form $H = \sum_{x,y,z} \tilde{eq}(r, x) \cdot \tilde{T}^{(1)}(x, y) \cdot \tilde{T}^{(2)}(x, y, z)$ by partially evaluating each operand on its degree-1 free indices using the verifier randomness and re-indexing the remaining variables so that shared dimensions align across operands. We then run a linear-time Sumcheck that extends prior linear-time provers [54] in two ways: it absorbs the extra $\tilde{eq}$ factor into each round's univariate polynomial, and it handles operands with different numbers of variables by viewing a shorter operand as a longer multilinear polynomial that ignores the extra coordinates. Due to the space constraints, we defer the complete prover algorithm to Sec. F.

4 Structured Lookups for Exponentiation

In this section, we introduce a method for proving exponentiation checks with structured lookups [41].

As discussed in Sec. 2.2, it is common to prove nonlinearities in LLMs via range and exponentiation checks. Recall from Sec. 2.5 that Shout [41] can prove a lookup relation with Eq. (5) using Sumcheck when the table is structured. It is well known that Shout can prove range check because the multilinear extension of the n -bit range

table T (the size of T is $N = 2^n$) is structured:

$$\tilde{T}(y) = \sum_{i=1}^n 2^{i-1} \cdot y_i. \quad (8)$$

However, exponentiation was previously considered to be unstructured [36], requiring $O(N)$ prover time for a size- N table in Eq. (5). In this section, we show how to instantiate exponentiation check in Shout using structured tables, such that our prover neither commits to any size- N auxiliary nor runs an $O(N)$ -time Sumcheck.

Classic unstructured exponentiation checks use a lookup table of the form

$$T = [\text{round}(2^s \cdot e^x) \mid x = Sq, \forall q \in \{0, -1, \dots, -N + 1\}],$$

where q is the quantized input, N is some public integer ($N = 2^n$) and S is the scale factor ($2^{-s} = S$) introduced in Sec. 2.3. The inputs x are non-positive because exponentiation is used only in Softmax (Eq. 1) and its related nonlinearities (e.g., Sigmoid [19], SiLU [12], and GeLU [22]). For presentation purposes, we set $s = 15$ below (typical values range from 10 to 16 in prior zkML systems [9, 36, 46]).

Constructing a structured multilinear polynomial from T evaluable in $O(\log N)$ time is non-trivial due to the presence of round and e^x , which is from floating-point arithmetic and are not directly compatible with field operations.

Warm-up intuition. Let $x = k \ln 2$. Then

$$T = [\text{round}(2^{s+k}) \mid k = \frac{Sq}{\ln 2}, \forall q \in \{0, -1, \dots, -N + 1\}].$$

Since k is generally not an integer, the rounding operation cannot be ignored. Nevertheless, this form suggests a promising direction because the power-of-2 table

$$T' = [2^y \mid y \in \{0, 1, \dots, N - 1\}]$$

is structured. In particular, its multilinear extension is

$$\tilde{T}'(y) = 2^{\sum_{i=1}^n 2^{i-1} \cdot y_i} = \prod_{i=1}^n ((2^{2^{i-1}} - 1) y_i + 1),$$

which can be evaluated in $O(\log N)$ time at any $y \in \mathbb{F}^n$, and is therefore compatible with the Shout lookup protocol [41]. The remaining challenge is reducing exponentiation to this form. We now introduce our method, denoted StructuredExp.

Exponent decomposition. Instead of writing $x = k \ln 2$, we decompose

$$q = k \cdot (-2^s \ln 2) + r,$$

where $k = \text{round}(\frac{-Sq}{\ln 2})$ is a non-negative integer, and $r = q + k \cdot (2^s \ln 2)$ satisfies $r \in [-\frac{2^s \ln 2}{2}, \frac{2^s \ln 2}{2}]$.

When $s = 15$, the integer k typically lies in the range $\{0, 1, \dots, 15\}$ for LLM inference workloads.⁶ If necessary, the prover can clamp q using ceiling and floor gates (which can be proved using range checks [10, 20]).

Constraints of StructuredExp decomposition. The prover commits to both k and q , and proves the following three constraints:

- (1) **Range check on k :** $k \in \{0, 1, \dots, 15\}$.
- (2) **Range check on r :** $r \in [-\frac{2^s \ln 2}{2}, \frac{2^s \ln 2}{2}]$.

⁶One can freely set the scale s larger (and the corresponding larger range for k) to ensure higher precision, here we set 15 for presentation purposes.

- (3) **Linear relation:** $r = q + k \cdot (2^s \ln 2)$, where $2^s \ln 2$ is a public constant.

These constraints ensure *soundness* because, for any committed input q , there exists exactly one valid pair (k, r) satisfying all three constraints. To see why, observe that the valid range for r has width exactly $2^s \ln 2$, which equals the “distance” between consecutive values of k in the linear relation. This partitions the real line into disjoint buckets of width $\ln 2$, each corresponding to a unique integer k . Consequently, a malicious prover cannot choose an incorrect k' : doing so would force $r' = q + k' \cdot (2^s \ln 2)$ to fall outside the valid range, causing the range check on r to fail.

The range check on r also guarantees that $|Sr| \leq \frac{\ln 2}{2} \approx 0.347$, ensuring the cubic Taylor approximation remains accurate (see below). We then write

$$e^x = e^{Sq} = e^{-k \ln 2} \cdot e^{Sr} = 2^{-k} \cdot e^{Sr},$$

and therefore the table entries in T become

$$\text{round}(2^s \cdot e^x) = \text{round}(2^{s-k} \cdot e^{Sr})$$

Let $y = s - k \in \{0, 1, \dots, 15\}$. Then the pow-of-two operation $2^{s-k} = 2^y$ can be proven using Shout lookup with the structured table T' . The remaining task is to approximate e^{Sr} .

Polynomial approximation. Since $Sr \in [-\frac{\ln 2}{2}, \frac{\ln 2}{2}]$, we can approximate e^{Sr} using a low-degree Taylor expansion:

$$\text{round}(2^{s-k} \cdot e^{Sr}) \approx 2^{s-k} \cdot \sum_{k=0}^d \frac{(Sr)^k}{k!}.$$

For presentation, here we use degree $d = 3$, yielding

$$\text{round}(2^{s-k} \cdot e^{Sr}) \approx 2^{s-k} \left[1 + Sr \left(1 + Sr \left(\frac{1}{2} + \frac{Sr}{6} \right) \right) \right].$$

Therefore, StructuredExp requires a power-of-2 table and four multiplications, both of which can be efficiently proven using a Shout lookup and EinSumcheck. The formal soundness argument reduces to the soundness of Shout [41] and EinSumcheck.

Below we analyze the effects of approximation on accuracy.

Approximation error analysis. We compare StructuredExp’s Taylor approximation against the exact lookup table on quantized integer inputs. For 100,000 random inputs $q \in [-340,695, 0]$ (corresponding to $x \in [-10.4, 0]$), we compute both the classic exponentiation entry $2^s e^{Sq}$ and the approximation $2^{s-k} \cdot \text{Taylor}(Sr)$.

Remarkably, **85% of entries match exactly** after rounding. Among the remaining entries, 88% have relative error below 0.01%, and 96% below 0.05%. Only 4% exceed 0.05%, corresponding to residuals near $\pm \frac{\ln 2}{2}$ where the Taylor approximation is least accurate. The mean absolute error is below 1 quantized unit. End-to-end evaluation (Table 1) shows negligible impact on model quality, increasing GPT-2 perplexity by only 0.01.

Error accumulation in Softmax. Let $D = \sum_{i=1}^{\ell} e^{x_i}$ and $\hat{D} = \sum_{i=1}^{\ell} \hat{e}(x_i)$ denote the exact and approximated Softmax denominators. A naive worst-case bound suggests $|\hat{D} - D| \leq \ell \times (5.66 \times 10^{-4}) \approx 2.3$ for context length $\ell = 4096$, assuming all terms incur the largest rare errors. In practice, approximation errors are approximately zero-mean. Across context lengths 128–4096 with 100 random trials each, the maximum denominator error at $n = 4096$ is only 0.0074, with relative error 0.015%.

Clamping impact on real attention patterns. The decomposition $q = k(-2^s \ln 2) + r$ requires $k \in \{0, \dots, 15\}$ for $s = 15$, corresponding to $q \geq -10.4 \cdot 2^s$. Smaller values must be clamped. To evaluate the practical impact, we analyzed GPT-2 attention scores after standard max-subtraction. All meaningful scores (softmax weight > 0.01) lie within $[-4.6, 0]$, well inside our valid range. The distribution has standard deviation $\sigma \approx 0.9$, with 83% of scores in $[-0.5, 0]$ and all scores above -5 . Consequently, no attention scores require clamping in practice for GPT-2. Larger models can further relax this threshold by increasing the scale s and range of k .

5 End-to-end Proof Generation for LLMs

In this section, we describe the core design of zkTransformer by presenting the building blocks used to arithmetize transformer layers and explaining how they compose into an end-to-end GKR proof for LLM inference. We begin with the security model (Sec. 5.1) and the end-to-end protocol (Sec. 5.2; full specification in Figure 2, Appendix H). We then introduce the building blocks (Sec. 5.3) and explain how each one is verified via a GKR sub-circuit. A concrete running example that chains these blocks together on a self-attention submodule is deferred to Appendix I, where we also include a snippet of Rust code (see List. 1) for it to demonstrate the improved usability of zkTransformer.

5.1 Security Model

We consider a setting where a *prover* (e.g., an ML-as-a-service provider or model owner) executes LLM inference on behalf of a *verifier* (e.g., a client, auditor, or regulator). The prover holds private witnesses: the quantized model weights $\{W^{(l)}\}$ (including the token-embedding matrix W_E), the auxiliary lookup matrices used by range and exponentiation checks (Sec. 5.3), and any witness tensors produced during inference. Any of these may be private; for example, the model weights may be proprietary to the prover and kept private. The verifier has the model input for inference and receives the inference result (i.e., the verifier has input/output token IDs) together with a cryptographic proof, and verifies that the result was correctly computed by the committed model without learning any of the prover’s private inputs beyond the output itself.

We assume the prover is computationally bounded and potentially malicious: it may attempt to return outputs inconsistent with the claimed model (e.g., a hand-crafted refusal, a censored generation, or a cheaper surrogate model’s output) while providing a convincing proof. The security goal is *soundness*: a cheating prover cannot produce an accepting proof for an output that is not the true argmax output of the claimed model on the claimed input, except with negligible probability. We also require *completeness* (an honest prover always convinces the verifier) and *zero-knowledge* (the proof reveals nothing about the private witnesses beyond the output token IDs). We realize zkTransformer without modifying GKR [56] (except for the arithmetization) or the polynomial commitment scheme KZH [18, 24], so standard security analyses apply. Furthermore, zero-knowledge can be achieved by combining blind-ing techniques for Sumcheck [48] with the zero-knowledge variant of KZH [18, 24], ensuring that all intermediate witnesses remain hidden while preserving soundness (Appendix D).

The verifier is assumed to be *semi-honest*, following the protocol while attempting to learn extra information from everything it observes. It has access to the public parameters: the LLM architecture (layer count L , hidden dimension, attention head count and head dimension, vocabulary size V), the quantization scale factors, any architecture-dependent public tensors (e.g., the causal mask, the precomputed RoPE cos/sin tensors, the structured range and power-of-2 tables of Sec. 4), the PCS verification key, and the input and output token ID transcripts. The verifier does *not* need access to the model weights, the token embeddings, or any intermediate hidden states.

5.2 End-to-end Protocol

The full protocol is presented in Figure 2 (Appendix H), including the commitment phase, Sumcheck proving for GKR and Shout, and evaluation proofs for the committed polynomials. Unlike prior work [36, 46] that only proves transformer blocks (including the causal attention mask [37]) on pre-embedded inputs, zkTransformer covers the full inference pipeline from input token IDs to the output token IDs. It includes (i) the token embedding, (ii) the learned positional encoding (for absolute-positional models such as GPT-2), (iii) all transformer blocks with mask (similar to prior work [36, 53]), (iv) the output projection head, and (v) the final argmax that selects the output token.

All five steps are expressed with the building blocks of Sec. 5.3: the embedding lookup and the output projection head reduce to invocations of the Einsum-expressible linear block; positional encoding and the attention masking (with the causal mask as a public operand) reduce to the element-wise addition block; and the argmax reduces to a composite gadget over Einsum, addition, and range check described in Sec. 5.3. A single GKR proof therefore attests that a committed model maps a public sequence of input token IDs to a public sequence of output token IDs, following the standard GKR structure [56] walking backward from output to input. Our implementation also includes recent techniques such as batch verification [53] and zkML polynomial evaluation [36].

5.3 Building Blocks

Each transformer layer is decomposed into a sequence of *building blocks*, where each block corresponds to a GKR sub-circuit (Sec. 2.4.2). The GKR protocol verifies each block via Sumcheck, and the evaluation claims produced at the end of one block’s Sumcheck serve as the input claims for the next block, chaining the blocks together from output to input. For convenience, we use $\tilde{v}$ to denote the multilinear extension of a tensor v . We describe each type below.

Einsum-expressible linear operations. Linear operations that can be written as an Einsum equation, including matrix multiplication, batched matrix multiplication, element-wise multiplication ($i, i \rightarrow i$), and axis summation, are verified with EinSumcheck (Theorem 3.1). Given an evaluation claim $\tilde{y}(r)$ on the output of the block, EinSumcheck arithmetizes the equation directly into a Sumcheck instance whose summand is a product of the input MLEs and a structured $\tilde{eq}$ factor. At the end of the Sumcheck, the verifier obtains evaluation claims on each input MLE at a fresh challenge, which are then forwarded to the next block. The $\tilde{eq}$ factor is evaluated directly by the verifier (Lemma 3.3), so no wiring-predicate

commitment is required. This block covers not only the attention and MLP projections inside each transformer layer, but also the token embedding lookup (expressed as $sv, vd \rightarrow sd$ with a one-hot selector S built from the public input token IDs) and the output projection head $bsd, vd \rightarrow bsv$.

Element-wise addition. Element-wise addition $y = a + b$ can be verified without a Sumcheck: because multilinear extension is linear, the verifier reduces an output claim $\tilde{y}(r)$ to claims on the operands by checking

$$\tilde{y}(r) = \tilde{a}(r) + \tilde{b}(r)$$

at the same challenge r (Schwartz–Zippel [40, 57]). The two claims on the operands are then forwarded to the upstream blocks. The same idea extends to any affine combination with public coefficients. This block is used to prove causal attention masking by combining with the attention logits in zkTransformer.

Free shape operations. Some shape operations require no proof in zkTransformer: they change only the semantic meaning of a tensor, not its multilinear extension. For instance, SQUEEZE removes a dimension of size 1 and UNSQUEEZE inserts one. Since these operations preserve the underlying values of the multilinear extensions, these operations introduce or remove no variables in the MLE, so the prover can pass the evaluation claim through unchanged.

Reshape. A reshape that factors a dimension $D_1 = D_2 \cdot D_3$ into a pair (D_2, D_3) (or vice versa) also requires no Sumcheck. Let $T \in \mathbb{F}^{D_1}$ and $T' \in \mathbb{F}^{D_2 \times D_3}$ be the tensors before and after the reshape. The prover reduces

$$\tilde{T}'(r_{D_2}, r_{D_3}) = \tilde{T}(r_{D_3}, r_{D_2}),$$

which the verifier accepts because both sides are multilinear and agree on the Boolean hypercube $\{0, 1\}^{\log D_2} \times \{0, 1\}^{\log D_3}$; by Lemma 2.2 they are the same polynomial. The claim on $\tilde{T}'$ is thus reduced to a claim on $\tilde{T}$ at a permuted challenge, which is forwarded upstream.

General permutation. A general permutation of a length- d tensor can always be expressed as multiplication by a Boolean permutation matrix $M \in \{0, 1\}^{d \times d}$ [8]. Given an output claim $\tilde{y}(r)$ where $y = Mx$, EinSumcheck reduces it via Sumcheck to

$$\tilde{y}(r) = \sum_z \tilde{M}(r, z) \tilde{x}(z).$$

A key observation is that for every structured permutation encountered in transformer models, $\tilde{M}$ admits a closed-form product of $\tilde{eq}$ factors. For example, the neighbor swap in RoPE for pairs $(t_1, t_2), (t_3, t_4), \dots, (t_{d-1}, t_d)$ has

$$\tilde{M}(w, z) = \tilde{eq}(w_1, 1 - z_1) \prod_{i=2}^{\log d} \tilde{eq}(w_i, z_i),$$

which the verifier can evaluate directly without a commitment to M [8]. The Sumcheck reduces the output claim to a single claim $\tilde{x}(z^*)$ on the unpermuted input, which is forwarded upstream.

Merging multiple claims on the same tensor. When a tensor v is consumed by more than one downstream block (e.g., a residual path), several evaluation claims on $\tilde{v}$ are produced during proof generation. We invoke the standard Sumcheck subroutine of [48, Sec. 4.5.2] to reduce k claims $\{\tilde{v}(r^{(1)}), \dots, \tilde{v}(r^{(k)})\}$ to a single claim

$\tilde{v}(\gamma)$ at a fresh challenge γ . The merged claim is then handled by the upstream block that produced v .

Rescale (rescaling after multiplication). After a multiplication, the scale factor doubles (Sec. 2.3). A Rescale block divides by 2^s : given operand claims $x_1 \approx 2^s \cdot x_2$, the prover writes

$$\tilde{x}_1(\gamma) = 2^s \cdot \tilde{x}_2(\gamma) + \tilde{res}(\gamma),$$

where the residual MLE $\tilde{res}$ is the advice witness and must take values in a small range (e.g., $[-2^{s-1}, 2^{s-1})$). The range check is provided by Shout [41]: the prover commits to a sparse Boolean indicator matrix a at the start of the GKR protocol, and verifier and prover run the Shout Sumcheck

$$\tilde{res}(\gamma) = \sum_{z \in \{0,1\}^s} \tilde{a}(\gamma, z) \tilde{T}_{\text{range}}(z),$$

where $\tilde{T}_{\text{range}}$ is the structured range-table MLE (Eq. 8). At the end, the verifier evaluates $\tilde{T}_{\text{range}}$ directly and the prover provides a PCS opening for $\tilde{a}$. Rescale is inserted after every Einsum-expressible multiplication that combines two scaled quantities.

Argmax. The final decoding step (i.e., asserting that the public output token t_i satisfies $t_i = \arg \max_j \text{logits}[i, j]$) is realized as a composition of previously introduced blocks rather than a dedicated sub-protocol. The prover commits a one-hot selector $S \in \{0, 1\}^{s \times V}$ satisfying $S[i, t_i] = 1$. Using the Einsum block $sv, sv \rightarrow s$, the prover extracts the selected logits $\text{sel}[i] = \text{logits}[i, t_i]$. The selected values are then broadcast and subtracted through the addition block to obtain $\text{diffs}[i, j] = \text{sel}[i] - \text{logits}[i, j]$. Finally, every entry of diffs is range-checked to encode a non-negative integer using range check. Soundness of S being one-hot is enforced by the gadget in Shout [41]. Together, these checks certify that $\text{logits}[i, t_i] \geq \text{logits}[i, j]$ for all j , and hence that t_i is a maximizer at each position, using only previously introduced components.

Exponentiation (e.g., Softmax, GeLU, and SiLU). As discussed in Sec. 2.2, all nonlinearities used in modern LLMs can be reduced to a combination of range checks and exponentiation checks using lookup protocols [29, 36, 46]. In zkTransformer, the exponentiation check is provided by StructuredExp (Sec. 4): given an output claim $\tilde{z}(r)$ on the exponentiation result and an input claim on $\tilde{x}$, StructuredExp reduces the claim via a Sumcheck over the structured power-of-2 table combined with a Taylor polynomial, without committing to any table-sized polynomial. Range checks reuse the Rescale block above. Because both the power-of-2 and range tables are structured, the verifier evaluates them directly (Eq. 8), and no lookup table ever needs to be committed.

A concrete end-to-end walk-through that chains these blocks on a single transformer submodule (the query projection with RoPE in LLaMA) is provided in Appendix I, along with the corresponding Rust code in our implementation to demonstrate high usability.

6 Experiments

In this section, we present experiments demonstrating the effectiveness of zkTransformer. We begin by describing the experimental setup, followed by performance comparisons with existing zkML systems. We then highlight the benefits of EinSumcheck and

StructuredExp. Finally, we analyze prover-time scalability and provide a breakdown of prover costs to guide future optimization.

6.1 Experimental Setup

We describe our experimental setup below.

Software. We implement zkTransformer primarily in Rust. We adopt recent work KZH-3 [18, 24] as the polynomial commitment scheme due to its linear-time prover and sublinear verification costs. Specifically, for a $(\log N)$ -variate multilinear polynomial, the prover runs in $O(N)$ time, while the proof size and verifier time are $O(N^{\frac{1}{3}})$. Finite-field arithmetic and elliptic-curve operations are implemented using the arkworks ecosystem. Following zkGPT [36], we choose the BN254 curve [4] as the underlying group, providing approximately 100-bit security.

Hardware. All experiments are conducted on a server with Intel(R) Xeon(R) Platinum 8358 CPU (32 cores at 2.60GHz) with 2TB memory. Unless otherwise stated, we enable 32 hardware threads for parallel execution as zkGPT [36] does.

Baselines. We compare our system against prior peer-reviewed or widely-adopted opensource ZKP frameworks for end-to-end LLM inference, including ZKML [10], EZKL [58], zkGPT [36], and zkLLM [46]. Among these baselines, general zkML frameworks including ZKML and EZKL provide AI-friendly interfaces (TFLite and ONNX, respectively), but incur high prover overhead. Due to the overhead, we compare the prover time for *single-token generation* with them. By contrast, the two Sumcheck-based systems, zkLLM and zkGPT, provide more reasonable prover and verifier overheads, but only support limited model architectures (e.g., zkGPT only supports the transformer block of GPT-2). Therefore, we compare with these systems using the amortized performance per generated token on GPT-2 under *multi-token generation*.

Since zkGPT also supports 32-threaded provers, we directly use the authors' implementation for evaluation. In contrast, zkLLM supports only GPU-based parallelism via CUDA. Accordingly, we report its performance on an NVIDIA A100 GPU and compare it with our results on a CPU-only server, following the evaluation methodology of zkGPT [36]. Specifically, an A100 has 19.5 TFLOPS and ≈ 2 TB/s memory bandwidth [31]; in contrast, our 32-core Xeon server has roughly $5.12 (= 32 * 0.16)$ TFLOPS and 200 GB/s bandwidth. Therefore, an NVIDIA A100 provides approximately 4 $\times$ higher compute and roughly 10 $\times$ higher memory bandwidth, making it substantially more efficient for dense tensor and polynomial operations common in zkML workloads.

6.2 Quantized Accuracy

We adopt fixed-point quantization with a single global scale parameter $s = 15$ similar to ZKML [10]. Each real value x is encoded as $q = \text{round}(2^{15} \cdot x)$. After each linear layer, outputs are rescaled to recover the original scale, ensuring that all subsequent layer inputs share a consistent scaling factor throughout the LLM inference.

Our global scale factor setting differs from some prior work (e.g., zkGPT sets the scale adaptively for each layer, which may leak weight distribution), so we evaluate the functional impact of quantization using perplexity (PPL) [23], a standard metric for LLM output quality in which lower values indicate better performance.

Table 1: Perplexity Before and After Quantization. Small perplexity gaps (< 0.5) indicate that quantization preserves model quality effectively [36].

Quantization	zkTransformer	zkGPT [36]
FP32 (Original)	25.17	29.30
Quantized model	25.18	29.50

Table 2: Amortized Per-Token Performance on GPT-2.

Framework	Prover Time	Verifier Time	Proof Size
zkGPT [36]	36.4 s	0.36 s	101 KB
zkLLM [46]	15.8 s	0.54 s	126 KB
zkTransformer	3.4 s	0.04 s	248 KB

To be consistent with zkGPT [36], we use the dataset WikiText-2. Following [36], when the increase in PPL is less than 0.5, it indicates that the functional degradation introduced by the quantization scheme is negligible, as shown in Table 1. Our experiments confirm that the fixed scale achieves sufficient accuracy: max relative error is 0.08% for individual exponentiations (Sec. 4), and the end-to-end perplexity increase is only 0.01 (Table 1), below the 0.5 threshold.⁷

6.3 Amortized Performance per Token

LLM applications [3, 32] typically require transformers [38, 50] to generate multiple tokens autoregressively. Therefore, for practical comparison with existing systems, we report the amortized performance per generated token. Specifically, we use 16 input prompt tokens and generate 16 output tokens for all evaluated models. For zkGPT, we use the released scripts to prove each generated token autoregressively. For zkLLM, we report the GPT-2 numbers provided by [36]. For zkTransformer, we report the amortized proving time obtained from end-to-end batch proof generation for multi-token GPT-2 inference. We note that the peak memory usage of zkTransformer is 50GB, compared to 194GB for zkGPT.

As shown in Table 2, zkTransformer achieves an amortized prover time of 3.4s per token, representing a 5–10 $\times$ speedup over prior Sumcheck-based frameworks. For the amortized verifier performance, zkTransformer achieves at least a 9 $\times$ speedup. Regarding the proof size, we attribute the gap primarily to (i) KZH-3's $O(N^{1/3})$ opening proofs (63.7% of the total proof; see Sec J) and (ii) end-to-end commitments and openings to lookup auxiliaries that prior systems omit in their implementation.

We emphasize that these speedups are conservative. Their released scripts [36, 45] omit several components required in practical transformer inference, such as the token embedding layer. In addition, the zkLLM measurements were obtained on an A100 GPU, this hardware-conservative comparison [36] indicates even after normalizing for compute, zkTransformer remains competitive.

⁷For LLaMA2-7B, we observe similar trend by setting $k \in \{0, 1, \dots, 31\}$ in StructuredExp and other scale $s = 15$ with the C4 dataset following [46]. The increase of the end-to-end PPL is 0.39 (< 0.5).

Table 3: Comparison with ZKML [10] on GPT-2.

Framework	Prover Time	Verifier Time	Proof Size
ZKML [10]	4915.7 s	13.89 s	27.47 KB
zkTransformer	93.66 s	0.93 s	4.31 MB

6.4 Performance of Single-Token Generation

We evaluate zkTransformer against general-purpose zkML frameworks based on Plonk-style proving systems [14], including ZKML [10] and EZKL [58]. We attempted to report amortized multi-token generation performance for these systems; however, the experiments were impractical due to either excessive memory consumption or prohibitively large prover runtimes. Since zkLLM [46] reports single-token generation results on LLaMA2-7B [50], we also implement LLaMA2-7B in zkTransformer for comparison.

For ZKML [10] (which uses the KZG [25] polynomial commitment scheme), we follow their evaluation setting and use an input sequence length of 64 tokens for GPT-2 inference while generating a single output token. As shown in Table 3, zkTransformer achieves a single-token prover time of 93.66s, representing more than a 50× speedup over ZKML. In addition, zkTransformer reduces verifier time by more than an order of magnitude. On the other hand, ZKML produces proofs that are approximately 160× smaller than those of zkTransformer. This difference stems from the constant-size proof property of KZG polynomial commitment scheme [25], whereas KZH-3 incurs $O(N^{1/3})$ proof size overhead.

For EZKL [58], we were unable to evaluate the 128M-parameter GPT-2 model due to excessive memory consumption of EZKL (exceeding 2 TB). Instead, we implement the 209K-parameter nanoGPT model used by default in EZKL within zkTransformer and compare single-token generation performance using an input sequence length of 1 token and generating 1 output token. As shown in Table 4, zkTransformer achieves approximately a 90× prover speedup and a 3× verifier speedup over EZKL. Similar to the ZKML comparison above, the proof size of zkTransformer is approximately 9× larger, reflecting the same tradeoff between the multilinear polynomial commitment scheme KZH and the univariate polynomial commitment scheme KZG [25].

Finally, we compare against the implementation of zkLLM [46]. We found that the released repository [45] primarily contains prover-side components rather than a complete end-to-end proving pipeline, so we report their aggregated prover time for single-token generation on an A100 GPU. As shown in Table 5, zkTransformer achieves approximately a 10× prover speedup while maintaining comparable verifier performance. Although zkLLM reports substantially smaller proof sizes, the released implementation does not appear to include several components required in practical transformer inference, such as token embedding layers and commitments (together with the corresponding opening proofs) for auxiliary lookup-related data. Consequently, the reported proof sizes of zkLLM may not fully reflect the complete end-to-end protocol overhead for LLaMA2-7B.

Table 4: Comparison with EZKL [58] on nanoGPT.

Framework	Prover Time	Verifier Time	Proof Size
EZKL [58]	19.94 s	0.22 s	40.80 KB
zkTransformer	0.23 s	0.06 s	356.33 KB

Table 5: Comparison with zkLLM [46] on LLaMA2-7B

Framework	Prover Time	Verifier Time	Proof Size
zkLLM [46]	617 s	2.39 s	220 KB
zkTransformer	59.2 s	2.44 s	12.62 MB

Table 6: Ablation Study of Linear and Nonlinear Arithmetization Strategies. (The symbol "✓" indicates the component remains the same as the one in zkTransformer.)

Linear	Lookup	Prover Time	Verifier Time	Proof Size
GKR	✓	5.33 s	0.06 s	294 KB
✓	LogUp	17.04 s	0.19 s	1607 KB
✓	✓	3.40 s	0.04 s	248 KB

6.5 Ablation Study

To understand how the proposed components, EinSumcheck and StructuredExp, contribute to the protocol performance, we ablate each component individually and measure the amortized per-token performance on GPT-2, as shown in Table 6.

Specifically, for the first experiment, we replace EinSumcheck with the classic GKR arithmetization (Eq. (4)) for all Einsum-expressible linear relations in GPT-2. Compared to the full zkTransformer system, this modification increases the amortized prover time from 3.40s to 5.33s per token and slightly increases the proof size from 248 KB to 294 KB. This result demonstrates that EinSumcheck substantially reduces the overhead of proving linear tensor operations by collapsing many intermediate relations into a single Sumcheck instance. Moreover, as we elaborate in the previous sections, EinSumcheck can improve both the developer experience and the completeness of the protocol.

For the second experiment, we replace StructuredExp and the Shout-based range proof [41] with the LogUp [17] lookup protocol used in prior work [36, 46] for exponentiation and range checks. This replacement increases the amortized prover time to 17.04s per token, increases the verifier time from 0.04s to 0.19s, and enlarges the proof size from 248 KB to 1607 KB. These results suggest that structured lookups are critical for efficiently handling nonlinear operations in transformer inference, particularly by reducing lookup-related commitment overhead.

6.6 Analysis of the Prover Time

We perform analyses for the prover time on GPT-2 to understand its scalability as the token sizes grow and to understand the prover time proportion that each component contributes to.

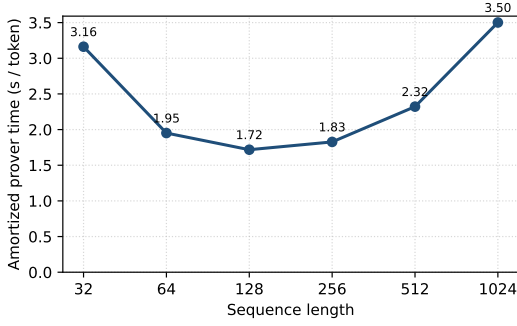


Figure 1: Amortized prover time per token on GPT-2 across sequence lengths {32, 64, 128, 256, 512, 1024}. The amortized cost is essentially flat (~ 1.8 s) up to context length 256 and then grows due to the quadratic attention cost and increasing memory pressure.

Scalability of the amortized prover time. We measure end-to-end prover time on a single batched proof for GPT-2 inference at sequence lengths {32, 64, 128, 256, 512, 1024} (the original GPT-2 supports up to 1024), fixing the input prompt to 16 tokens and amortizing over the remaining generated tokens. For each run, we report the amortized prover time per generated token: we exclude the one-time cost of committing to the model weights and include only the witness commitment and the Sumcheck proving cost.

Figure 1 shows that the amortized prover time per token stays below 3.5 s across all evaluated sequence lengths and reaches a minimum of ~ 1.7 s in the mid-range. The slightly higher cost at sequence length 32 (3.16 s) reflects the small denominator (only 16 generated tokens), where per-batch fixed overheads are incompletely amortized. Beyond context length 256, the amortized cost rises modestly to 2.3 s (512) and 3.5 s (1024), reflecting the quadratic scaling of attention. We note that this configuration is also limited by memory: at sequence length 1024, the resident set size of the prover exceeds 1 TB, indicating that further scaling is dominated by memory overhead rather than purely compute. We leave the optimization to the future work.

Prover-time breakdown. Table 7 reports the breakdown of the total prover time at sequence length 32 across five components: *commit-to-advice* (committing to non-constant witness/auxiliary polynomials), *opening* (polynomial commitment opening proofs), *lookup* (range and StructuredExp lookups), *linear* (mainly sumchecks for EinSumcheck), and *others* (a few miscellaneous shape operations). The dominant components are *lookup* (36.6%) and *commit-to-advice* (31.5%), which together account for 68% of the prover cost. Both originate from the same underlying source: the size of the auxiliary tensors that the prover must commit to and prove range constraints over. This points to a clear direction for future work: further reducing the advice produced by nonlinearity gadgets would directly cut both buckets simultaneously. In contrast, the linear Sumcheck portion (23.7%) is already near the linear-time lower bound enabled by EinSumcheck, and the opening cost (6.3%) is small thanks to the KZH-3 commitment scheme [24].

Table 7: Total Prover Time Breakdown at Sequence Length 32 on GPT-2.

Component	Time (s)	%
Commit to advice	15.93	31.5
Opening	3.19	6.3
Lookup (range + StructuredExp)	18.52	36.6
Linear (EinSumcheck + Add)	12.00	23.7
Others	0.96	1.9
Total	50.59	100.0

7 Related Work

Arithmetization for Sumcheck-based protocols. The classic GKR protocol supports general arithmetic circuits via explicit addition and multiplication gates [15]. Zhang et al. [56] extend GKR to allow wiring from arbitrary previous layers, but still rely on wiring predicates to enforce gate constraints. Power circuits [43] further simplify wiring for addition and multiplication, yet still require commitments to wiring predicates for succinctness. In comparison, MatMul [47] shows that specialized arithmetization for matrix multiplication can achieve asymptotically optimal prover time compared to general-purpose GKR. Our EinSumcheck follows this specialization philosophy: instead of supporting arbitrary circuits, it targets the linear relations commonly appearing in ML models. EinSumcheck generalizes MatMul and supports a broad class of Einsum-expressible tensor contractions using a single Sumcheck instance, eliminating wiring predicates entirely.

ZKP for LLM inference. EZKL [58], a general-purpose zkML framework, supports proving for small LLMs such as nanoGPT. ZKML [10] demonstrated the first end-to-end proof system for GPT-2 inference (128M params), but required over an hour per forward pass, revealing significant scalability challenges. zkLLM [46] targets attention layers using GPU-parallelized provers, but does not provide a general end-to-end framework and depends on high-end GPU accelerators. zkPyTorch [53] demonstrated the idea of batch verification for multi-token generation, but did not release the implementation. More recently, zkGPT [36] proposed a GKR-based proof system for GPT-2 that integrates optimized matrix multiplication, advice-based nonlinear verification, and extensive circuit-level optimizations, achieving prover times on the order of tens of seconds on CPUs. However, zkGPT is specialized to GPT-2 and relies on manual, model-specific circuit engineering. In contrast, our work targets general-purpose, compiler-driven arithmetization for transformer inference based on Sumcheck and GKR. By combining Einsum-based arithmetization (EinSumcheck) with structured nonlinear checks (StructuredExp), zkTransformer supports multiple LLM architectures without model-specific circuit design while achieving competitive end-to-end performance.

Concurrent work zkAgent [52] demonstrates promising prover performance using their optimized token-to-embedding lookups together with batch verification [53] and Shout range check [41]. Their techniques are orthogonal to EinSumcheck and StructuredExp, and we believe integrating them could further improve our prover performance. We leave this direction to future work.

8 Conclusion

We present zkTransformer, an end-to-end ZKP framework for verifiable LLM inference. EinSumcheck lifts linear tensor operations to an Einsum abstraction and compiles them into a single Sumcheck instance, removing manual circuit engineering and a class of completeness bugs in prior implementations. StructuredExp arithmetizes exponentiation using only structured lookup tables, eliminating the table-sized commitments that dominate LogUp-style protocols. On GPT-2, zkTransformer achieves an amortized prover time of 3.4 seconds per token, a 5–10× speedup over state-of-the-art systems zkLLM and zkGPT.

References

- [1] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) (*OSDI'16*). USENIX Association, USA, 265–283.
- [2] Anonymous. 2026. Implementation for zkTransformer. <https://anonymous.4open.science/r/zkes-49F4>.
- [3] Anthropic. 2025. Claude 3.5 Sonnet. <https://claude.ai>. Accessed: 2026-01-28.
- [4] Paulo S. L. M. Barreto and Michael Naehrig. 2005. Pairing-Friendly elliptic curves of prime order. In *Proceedings of the 12th International Conference on Selected Areas in Cryptography* (Kingston, ON, Canada) (*SAC'05*). Springer-Verlag, Berlin, Heidelberg, 319–331. doi:10.1007/11693383_22
- [5] Benedikt Bünz, Jessica Chen, and Zachary DeStefano. 2025. Linear*-Time Permutation Check. Cryptology ePrint Archive, Paper 2025/1850.
- [6] Sarah H. Cen and Rohan Alur. 2024. From Transparency to Accountability and Back: A Discussion of Access and Evidence in AI Auditing. In *Proceedings of the 4th ACM Conference on Equity and Access in Algorithms, Mechanisms, and Optimization* (San Luis Potosi, Mexico) (*EAAMO '24*). Association for Computing Machinery, New York, NY, USA, Article 13, 14 pages. doi:10.1145/3689904.3694711
- [7] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. Springer-Verlag, Berlin, Heidelberg, 499–530. doi:10.1007/978-3-031-30617-4_17
- [8] Bing-Jyue Chen, Lilia Tang, David Heath, and Daniel Kang. 2025. Linear-time and Logarithmically-sound Permutation and Multiset SNARKs. Cryptology ePrint Archive, Paper 2025/1967. <https://eprint.iacr.org/2025/1967>
- [9] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. 2025. ZKTorch: Compiling ML Inference to Zero-Knowledge Proofs via Parallel Proof Accumulation. arXiv:2507.07031 [cs.CR] <https://arxiv.org/abs/2507.07031>
- [10] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 560–574.
- [11] Albert Einstein. 1916. The Foundation of the General Theory of Relativity. *Annalen der Physik* 49, 7 (1916), 769–822. doi:10.1002/andp.19163540702
- [12] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2017. Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning. arXiv:1702.03118 [cs.LG] <https://arxiv.org/abs/1702.03118>
- [13] Amos Fiat and Adi Shamir. 1987. How To Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO'86*, Andrew M. Odlyzko (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 186–194.
- [14] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Paper 2019/953.
- [15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2015. Delegating Computation: Interactive Proofs for Muggles. *J. ACM* 62, 4, Article 27 (Sept. 2015), 64 pages. doi:10.1145/2699436
- [16] Angus Gruen. 2024. Some Improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108. <https://eprint.iacr.org/2024/108>
- [17] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530.
- [18] Hossein Hafezi, Alireza Shirzad, Benedikt Bünz, and Joseph Bonneau. 2025. IronDict: Transparent Dictionaries from Polynomial Commitments. Cryptology ePrint Archive, Paper 2025/1580.
- [19] Jun Han and Claudio Moraga. 1995. The Influence of the Sigmoid Function Parameters on the Speed of Backpropagation Learning. In *Proceedings of the International Workshop on Artificial Neural Networks: From Natural to Artificial Neural Computation (IWANN '96)*. Springer-Verlag, Berlin, Heidelberg, 195–201.
- [20] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. 2025. Scalable Zero-knowledge Proofs for Non-linear Functions in Machine Learning. Cryptology ePrint Archive, Paper 2025/507. <https://eprint.iacr.org/2025/507>
- [21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385 [cs.CV] <https://arxiv.org/abs/1512.03385>
- [22] Dan Hendrycks and Kevin Gimpel. 2023. Gaussian Error Linear Units (GELUs). arXiv:1606.08415 [cs.LG] <https://arxiv.org/abs/1606.08415>
- [23] Frederick Jelinek, Robert L. Mercer, Lalit R. Bahl, and Janet M. Baker. 1977. Perplexity—a measure of the difficulty of speech recognition tasks. *Journal of the Acoustical Society of America* 62 (1977). <https://api.semanticscholar.org/CorpusID:121680873>
- [24] George Kadianakis, Arantxa Zapico, Hossein Hafezi, and Benedikt Bünz. 2025. KZH-Fold: Accountable Voting from Sublinear Accumulation. Cryptology ePrint Archive, Paper 2025/144.
- [25] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology – ASIACRYPT 2010*, Masayuki Abe (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 177–194.
- [26] Tohru Kohrita and Patrick Towa. 2023. Zeromorph: Zero-Knowledge Multilinear-Evaluation Proofs from Homomorphic Univariate Commitments. Cryptology ePrint Archive, Paper 2023/917. doi:10.1007/s00145-024-09519-0
- [27] Jonathan Lee. 2021. Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments. In *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part II* (Raleigh, NC, USA). Springer-Verlag, Berlin, Heidelberg, 1–34. doi:10.1007/978-3-030-90453-1_1
- [28] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. ZkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2968–2985.
- [29] Tao Lu, Haoyu Wang, Wenjie Qu, Zonghui Wang, Jinye He, Tianyang Tao, Wenzhi Chen, and Jiaheng Zhang. 2024. An Efficient and Extensible Zero-knowledge Proof Framework for Neural Networks. Cryptology ePrint Archive, Paper 2024/703. <https://eprint.iacr.org/2024/703>
- [30] C. Lund, L. Fortnow, H. Karloff, and N. Nisan. 1990. Algebraic methods for interactive proof systems. In *Proceedings [1990] 31st Annual Symposium on Foundations of Computer Science*. 2–10 vol.1. doi:10.1109/FSCS.1990.89518
- [31] NVIDIA. 2024. NVIDIA A100 Tensor Core GPU Datasheet. <https://www.nvidia.com/en-us/data-center/a100/>. Accessed: 2026-04-26.
- [32] OpenAI. 2023. ChatGPT. <https://chat.openai.com/chat>. Accessed: 2026-01-28.
- [33] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.
- [34] Perplexity AI, Inc. 2025. Perplexity AI. <https://www.perplexity.ai/>. Accessed: 2026-01-28.
- [35] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) (*CCS '23*). Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157
- [36] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: an efficient non-interactive zero-knowledge proof framework for LLM inference. USENIX Association, USA.
- [37] Alec Radford and Karthik Narasimhan. 2018. Improving Language Understanding by Generative Pre-Training. <https://api.semanticscholar.org/CorpusID:49313245>
- [38] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. (2019).
- [39] Tim Rocktäschel. 2018. Einsum is All You Need – Einstein Summation in Deep Learning. <https://web.archive.org/web/20250514130020/https://rockt.ai/2018/04/30/einsum>. Accessed: 2025-06-25.
- [40] J. T. Schwartz. 1980. Fast Probabilistic Algorithms for Verification of Polynomial Identities. *J. ACM* 27, 4 (Oct. 1980), 701–717. doi:10.1145/322217.322225
- [41] Srinath Setty and Justin Thaler. 2025. Twist and Shout: Faster memory checking arguments via one-hot addressing and increments. Cryptology ePrint Archive, Paper 2025/105.
- [42] Srinath Setty, Justin Thaler, and Riad Wahby. 2024. Unlocking the Lookup Singularity with Lasso. In *Advances in Cryptology – EUROCRYPT 2024: 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26–30, 2024, Proceedings, Part VI* (Zurich, Switzerland). Springer-Verlag, Berlin, Heidelberg, 180–209. doi:10.1007/978-3-031-58751-1_7

- [43] Lev Soukhanov. 2023. Power circuits: a new arithmetization for GKR-styled sumcheck. Cryptology ePrint Archive, Paper 2023/1611. <https://eprint.iacr.org/2023/1611>
- [44] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomput.* 568, C (Feb. 2024), 12 pages. doi:10.1016/j.neucom.2023.127063
- [45] Haochen Sun. 2024. Implementation for zkLLM. <https://github.com/jvhs0706/zkllm-ccs2024>. The prefix of commit hash: 993311e.
- [46] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4405–4419.
- [47] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology – CRYPTO 2013*, Ran Canetti and Juan A. Garay (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 71–89.
- [48] Justin Thaler. 2022. Proofs, Arguments, and Zero-Knowledge. *Foundations and Trends in Privacy and Security* 4, 2–4 (12 2022), 117–660.
- [49] Justin Thaler et al. 2022. Proofs, arguments, and zero-knowledge. *Foundations and Trends® in Privacy and Security* 4, 2–4 (2022), 117–660.
- [50] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (*NIPS'17*). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.
- [52] Lizheng Wang, Hancheng Lou, Chongrong Li, Yu Yu, and Yuncong Hu. 2026. zkAgent: Verifiable Agent Execution via One-Shot Complete LLM Inference Proof. Cryptology ePrint Archive, Paper 2026/199. <https://eprint.iacr.org/2026/199>
- [53] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. 2025. zkPyTorch: A Hierarchical Optimized Compiler for Zero-Knowledge Machine Learning. Cryptology ePrint Archive, Paper 2025/535. <https://eprint.iacr.org/2025/535>
- [54] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. 2019. Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation. In *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III* (Santa Barbara, CA, USA). Springer-Verlag, Berlin, Heidelberg, 733–764. doi:10.1007/978-3-030-26954-8_24
- [55] Yunbo Yang, Yuejia Cheng, Haibo Tang, Guomin Yang, Bingsheng Zhang, and Kui Ren. 2026. SoK: Understanding zkVM: From Research to Practice. Cryptology ePrint Archive, Paper 2026/525. <https://eprint.iacr.org/2026/525>
- [56] Jiaheng Zhang, Tianyi Liu, Weijie Wang, Yinuo Zhang, Dawn Song, Xiang Xie, and Yupeng Zhang. 2021. Doubly Efficient Interactive Proofs for General Arithmetic Circuits with Linear Prover Time. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (*CCS '21*). Association for Computing Machinery, New York, NY, USA, 159–177. doi:10.1145/3460120.3484767
- [57] Richard Zippel. 1979. Probabilistic algorithms for sparse polynomials. In *Proceedings of the International Symposium on Symbolic and Algebraic Computation (EUROSAM '79)*. Springer-Verlag, Berlin, Heidelberg, 216–226.
- [58] Zkconduit Inc. 2023. The EZKL System. <https://docs.ezkl.xyz/>

A Ethical Considerations

This work studies zero-knowledge proofs for large language model (LLM) inference, with the goal of enabling verifiable execution of proprietary models without revealing model parameters or internal intermediate states. While zkTransformer is primarily a systems and cryptography contribution, its deployment may affect multiple stakeholders and raises ethical considerations related to transparency, misuse, access asymmetry, and accountability.

A.1 Stakeholders

Model providers. Model owners benefit from zkTransformer because it enables cryptographic guarantees of faithful inference without disclosing proprietary model weights or architecture details. This can strengthen trust with users and regulators while preserving intellectual property. However, lowering the cost of verifiable inference may increase pressure on providers to offer

proofs even when they would prefer not to. Participation remains voluntary, and our system does not weaken existing confidentiality guarantees.

Users and regulators. Users, auditors, and regulators benefit from stronger assurance that an output was produced by a specific advertised model rather than a cheaper approximation or manipulated computation. This is particularly relevant in high-stakes settings such as compliance auditing. Unlike traditional auditing approaches that require access to model weights, zkTransformer preserves model confidentiality and avoids exposing sensitive parameters.

Researchers and developers. By providing an AI-friendly interface based on Einsum expressions and an open-source implementation, this work lowers the barrier to experimenting with verifiable LLM inference. At the same time, this increases the importance of responsible benchmarking and clear communication of system limitations to avoid overstating real-world readiness or security guarantees.

Potential adversaries. As with many cryptographic efficiency improvements, there is a risk that adversaries could misuse inference correctness proofs to make dishonest services appear trustworthy. Our system proves only that a specified computation was executed correctly; it does not guarantee that a model is safe, unbiased, or trained on appropriate data. We explicitly caution against interpreting inference proofs as broader guarantees of model trustworthiness.

A.2 Risks and Mitigations

Over-interpretation. There is a risk that users may conflate zero-knowledge proofs of inference correctness with guarantees about model alignment, fairness, or safety. To mitigate this, we clearly scope our claims to inference-level correctness and do not make claims about training data or downstream behavior.

Approximation and numerical error. zkTransformer relies on fixed-point quantization and low-degree polynomial approximations for certain nonlinear functions. While our experiments show that approximation errors are negligible at commonly used precision levels, these approximations may introduce small numerical deviations. We explicitly document quantization parameters and approximation bounds to allow users to assess suitability for their applications.

Resource asymmetry. Although zkTransformer significantly reduces prover overhead compared to prior work, generating proofs for large LLMs still requires substantial computational resources. This may advantage well-resourced organizations. We partially mitigate this by focusing on asymptotic improvements, CPU-based evaluation, and open-source release to support broader access and independent verification.

A.3 Data and Human Subjects

This work does not involve human subjects, personal data, or user-generated content. All experiments are conducted on standard benchmark models using synthetic or publicly available inputs.

A.4 Responsible Disclosure and Use

We release zkTransformer as open source to support reproducibility and community scrutiny. We encourage practitioners to treat verifiable inference as one component of a broader trust framework rather than as a standalone guarantee of ethical or safe model behavior.

A.5 Ethical Justification

We believe the benefits of enabling verifiable LLM inference without compromising model confidentiality outweigh the potential risks. By strengthening accountability while preserving intellectual property and privacy, zkTransformer advances responsible deployment of large-scale AI systems.

B Open Science

In this section, we list all artifacts necessary to evaluate the contributions of this paper.

We release the implementation of zkTransformer at [2]. The proving scripts for LLMs are located under `src/bin/`. The PPL evaluation scripts are provided under `ppl_repro/`, following the methodology of zkLLM [45, 46].

We use standard transformer models and datasets from Hugging Face (<https://huggingface.co/>). Our evaluation includes both LLaMA2-7B (meta-llama/Llama-2-7b-hf) [50] and GPT-2 Small (openai-community/gpt2) [38]. Perplexity is measured on the WikiText-2 test set (mindchain/wikitext2) and the C4 validation set (allenai/c4) using strided sliding-window evaluation with stride 512 and context length 1024 (4096 for LLaMA2). The increase of perplexity on WikiText-2 for GPT-2 is 0.01 and the increase of perplexity on C4 for LLaMA2-7B is 0.39 (i.e., both are < 0.5), which indicates the effectiveness of the quantization.

The implementations of the baseline systems are available at the following links:

- (1) ZKML [10]: <https://github.com/uiuc-kang-lab/zkml.git>
- (2) zkLLM [46]: <https://github.com/jvhs0706/zkllm-ccs2024.git>
- (3) zkGPT [36]: <https://zenodo.org/records/16958213>

C Generative AI Usage

Generative AI tools (Claude by Anthropic) were used in preparing this manuscript to assist with drafting and editing text, as well as supporting implementation of the evaluation code. All AI-assisted content was carefully reviewed, validated for correctness, and revised by the authors. All technical contributions, theoretical results, experimental design, and evaluation were conducted by the authors.

D Cryptographic Background

In this section, we briefly review the theories behind most ZKP protocols.

D.1 Interactive proofs and arguments of knowledge

An interactive proof is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$. The protocol runs in several rounds, allowing $\mathcal{V}$ to ask questions in each round based on $\mathcal{P}$'s answers in previous

rounds. We formalize this in terms of $\mathcal{P}$ trying to convince $\mathcal{V}$ that $C(x) = y$, and give the formal definitions below.

Let C be a function. A pair of interactive machines $\langle \mathcal{P}, \mathcal{V} \rangle$ is an interactive proof for C with soundness ϵ if the following holds:

- Completeness. For every x such that $C(x) = y$ it holds that $\Pr[\langle \mathcal{P}, \mathcal{V} \rangle(x) = \text{accept}] = 1$.
- ϵ -Soundness. For any x with $C(x) \neq y$ and any $\mathcal{P}^*$ it holds that $\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle = \text{accept}] \leq \epsilon$

We say an interactive proof scheme has succinct proof size and verifier time if they are $O(\text{polylog}(|C|, |x|))$.

When the prover is computationally bounded, the protocol is called an *argument*. An argument additionally satisfies *knowledge soundness* if any prover that convinces the verifier can be efficiently transformed into an extractor that recovers a valid witness.

D.2 Polynomial interactive oracle proofs (PIOPs)

Modern zkSNARKs are typically built from *polynomial interactive oracle proofs* (PIOPs). A PIOP is a public-coin (i.e., where all random choices made by the verifier are revealed to the prover) interactive proof for an *oracle relation*, where the prover sends polynomial oracles and the verifier queries these oracles at randomly chosen points. The verifier's checks reduce global correctness of polynomial relations to consistency at a small number of random evaluations. PIOPs are information-theoretic objects: the verifier can query oracle polynomials directly, and soundness holds against unbounded provers. Many zkML protocols such as [28, 36] can be naturally expressed as PIOPs over multilinear polynomials. Importantly, soundness of a PIOP also implies knowledge soundness [7].

D.3 Polynomial commitment schemes (PCS)

A polynomial commitment scheme (PCS) allows a prover to commit to a polynomial f and later prove, that a claimed value y equals $f(z)$ for a verifier-chosen point z .

Recent PCSs support multilinear polynomials with linear prover time and sublinear (often polylogarithmic) verifier time in the size of the evaluation domain. In this work, we primarily use KZH [18, 24].

D.4 From PIOPs to zkSNARKs via compilation

A PIOP can be compiled into an interactive argument of knowledge by replacing each polynomial oracle with a commitment and each oracle query with a PCS evaluation proof. The verifier accepts if both the original PIOP checks and all PCS verification checks succeed. If the PCS satisfies witness-extended emulation, then the compiled protocol preserves knowledge soundness [7]. Moreover, if the PCS is hiding and its evaluation protocol is honest-verifier zero knowledge, then the compiled argument is also zero knowledge.

Because PIOPs are public-coin, the resulting interactive argument can be made non-interactive using the Fiat-Shamir transform [13], yielding a non-interactive argument of knowledge (NARK). When both proof size and verifier time are sublinear in the witness size, the result is a *succinct* NARK, i.e., a SNARK. Overall efficiency is determined jointly by the algebraic structure of the PIOP and the concrete performance of the underlying PCS.

D.5 Zero-Knowledge Techniques

To achieve zero-knowledge, the proof must reveal nothing about the prover's private inputs beyond the computation's output. This requires two well-known blinding mechanisms.

Hiding polynomial commitments. A standard PCS commitment $C = \text{Commit}(f)$ may reveal information about the polynomial f . To make the commitment *hiding*, the KZH scheme [18, 24] adds a random masking term: $C = \text{Commit}(f) + \rho \cdot h$, where h is a random group element included in the public parameters and ρ is a fresh blinding scalar chosen by the prover. The opening proof is adjusted to account for the mask, so the verifier can still check correctness. Since ρ is uniformly random, the commitment C is statistically independent of f .

Masked sumcheck. In the sumcheck protocol, each round polynomial $f_i(x_i)$ sent by the prover may leak information about the witness. To prevent this, the prover adds a low-degree random masking polynomial to each round polynomial [49]. Specifically, before the protocol begins, the prover commits to a random polynomial g of the same degree as f ; in each round, the prover sends $f_i(x_i) + g_i(x_i)$ instead of $f_i(x_i)$, and the verifier adjusts the check using the commitment to g . This ensures that the transcript is statistically indistinguishable from random while preserving completeness and soundness.

Together, hiding commitments and masked sumcheck transcripts yield a zero-knowledge proof system.

D.6 Implications for zkTransformer

Our system follows this standard pipeline: we use a standard PIOP (i.e., the GKR protocol [15, 28] and the Shout lookup [41]) that captures transformer inference using multilinear polynomial relations, and we instantiate the oracles with KZH PCS [18, 24] to obtain a SNARK. To turn zkTransformer into a zkSNARK, the blinding techniques described in Sec. D.5 (i.e., hiding KZH commitments [18, 24] and masked sumcheck rounds [48, 49]) are directly applicable to zkTransformer without modifying its GKR structure or its EinSumcheck/StructuredExp sub-protocols.

E The Sumcheck Protocol

Here we show the full Sumcheck protocol [30].

Protocol B: Sumcheck protocol [7, 30]

Goal: Given a polynomial f and a claim sum H for μ -variate degree d polynomial $f \in \mathcal{F}_\mu^{(\leq d)}$, prove that $H = \sum_{x \in \{0,1\}^\mu} f(x)$. The protocol proceeds in μ rounds:

Round 1: $\mathcal{P}$ sends univariate polynomial

$$f_1(x_1) := \sum_{b_2, \dots, b_\mu \in \{0,1\}} f(x_1, b_2, \dots, b_\mu)$$

$\mathcal{V}$ checks $H = f_1(0) + f_1(1)$ and sends random challenge $r_1 \in \mathbb{F}$.

Round i (for $2 \leq i \leq \mu - 1$): $\mathcal{P}$ sends univariate polynomial

$$f_i(x_i) := \sum_{b_{i+1}, \dots, b_\mu \in \{0,1\}} f(r_1, \dots, r_{i-1}, x_i, b_{i+1}, \dots, b_\mu)$$

$\mathcal{V}$ checks $f_{i-1}(r_{i-1}) = f_i(0) + f_i(1)$ and sends random challenge $r_i \in \mathbb{F}$.

Round μ : $\mathcal{P}$ sends univariate polynomial

$$f_\mu(x_\mu) := f(r_1, r_2, \dots, r_{\mu-1}, x_\mu)$$

$\mathcal{V}$ checks $f_{\mu-1}(r_{\mu-1}) = f_\mu(0) + f_\mu(1)$, sends random challenge $r_\mu \in \mathbb{F}$, and outputs 1 if $f_\mu(r_\mu) = f(r_1, r_2, \dots, r_\mu)$ by asking $\mathcal{P}$ to provide evaluation proof for $f(r_1, r_2, \dots, r_\mu)$.

F The Main Prover Algorithm for EinSumcheck

Since zkTransformer only requires the two-operand case, we focus on that setting in this subsection; the extension to more operands is straightforward. Consider the Einsum expression $\text{bsd}h\nu, \text{btd}h \rightarrow \text{bst}h$. A EinSumcheck prover will show that the claimed output $\tilde{Z}(r_b, r_s, r_t, r_h)$ equals

$$\sum_{(b,d,h,v) \in \{0,1\}^U} \tilde{eq}(r_b, b) \cdot \tilde{eq}(r_h, h) \cdot \tilde{A}(b, r_s, d, h, v) \cdot \tilde{B}(b, r_t, d, h), \quad (9)$$

given the verifier challenge $r = (r_b, r_s, r_t, r_h)$, where $U = \log B + \log D + \log H + \log V$. Here $A \in \mathbb{F}^{B \times S \times D \times H \times V}$ and $B \in \mathbb{F}^{B \times T \times D \times H}$. This is not directly in the standard form supported by existing linear-time Sumcheck provers [54].

To address this issue, we propose a new linear-time prover algorithm that extends prior linear-time Sumcheck provers.

A canonical form. We design a linear-time Sumcheck prover for relations of the following form:

$$H = \sum_{x,y,z} \tilde{eq}(r, x) \cdot \tilde{T}^{(1)}(x, y) \cdot \tilde{T}^{(2)}(x, y, z), \quad (10)$$

which all two-operand EinSumcheck relations can be organized into. We use Eq. (9) as a concrete example.

Organizing (9) into (10). Before invoking Sumcheck, the prover partially evaluates each operand using the verifier randomness on degree-1 free indices. Let H denote the claimed output evaluation (e.g., $H = \tilde{Z}(r_b, r_s, r_t, r_h)$), and define multilinear polynomials over the remaining variables:

$$\tilde{T}^{(2)}(b, d, h, v) := \tilde{A}(b, r_s, d, h, v), \quad \tilde{T}^{(1)}(b, d, h) := \tilde{B}(b, r_t, d, h).$$

Then (9) becomes H equals

$$\sum_{(b,d,h,v) \in \{0,1\}^U} \tilde{eq}(r_b, b) \cdot \tilde{eq}(r_h, h) \cdot \tilde{T}^{(2)}(b, d, h, v) \cdot \tilde{T}^{(1)}(b, d, h).$$

We use three simple but useful facts.

- (1) **Variable order in Sumcheck is flexible.** Sumcheck does not require eliminating variables in any fixed order (e.g., left-to-right). Any order is valid as long as both the initial sum is correct, the verifier knows the order, and the final evaluation passes.
- (2) **Lagrange basis factors.** By definition, $\tilde{eq}(r_b, b) \cdot \tilde{eq}(r_h, h) = \tilde{eq}((r_b, r_h), (b, h))$.
- (3) **Multilinear polynomials can be re-indexed.** If $\tilde{T}(b, d, h)$ is multilinear and we define $\tilde{T}'(b, h, d)$ by permuting its input coordinates, then $\tilde{T}$ and $\tilde{T}'$ represent the same multilinear function under a reordering of variables (i.e., $\tilde{T}(r_b, r_d, r_h) = \tilde{T}'(r_b, r_h, r_d)$, $\forall r_b, r_d, r_h \in \mathbb{F}^*$).

Using these facts, we reorder variables to eliminate in the order $b \rightarrow h \rightarrow d \rightarrow v$, and rewrite the relation as H equals

$$\sum_{(b,h,d,v) \in \{0,1\}^U} \tilde{eq}((r_b, r_h), (b, h)) \cdot \tilde{T}'^{(1)}(b, h, d) \cdot \tilde{T}^{(2)}(b, h, d, v),$$

where $U = \log B + \log H + \log D + \log V$ and $\tilde{T}^{(i)}$ is obtained from $\tilde{T}^{(i)}$ by reordering its input variables accordingly. Let $x = (b, h)$, $y = d, z = v$, and $r = (r_b, r_h)$. The above matches the canonical form in Eq. (10).

Linear-time prover for (10). We now describe how to prove Eq. (10) in linear time. In our algorithm, we use a proxy procedure⁸ adapted from [54]. With the proxy algorithm, our main prover algorithm is shown in Alg. 1. Its running time scales linearly with the witness size, i.e., $O(2^{\ell+\mu+\eta})$. Concretely, the proxy FUNCTIONEVALUATIONS runs in $O(2^\mu)$ time for a μ -variable polynomial [54], and the overall cost is dominated by the largest operand $\tilde{T}^{(2)}$, which has $\ell + \mu + \eta$ variables. Alg. 1 additionally performs $O(2^{\ell+\mu+\eta})$ field multiplications, resulting in total prover time $O(2^{\ell+\mu+\eta})$.

The algorithm follows the structure of standard linear-time Sumcheck provers, but additionally handles the case where the operands have different numbers of variables. It relies on the fact that an ℓ -variate multilinear polynomial $\tilde{f}(x)$ can be viewed as an $(\ell + \mu)$ -variate multilinear polynomial $f(x, y)$ that ignores y . This is the reason why the modulo indexing appears in Alg. 1.

Finally, note that Alg. 1 is a generalized prover algorithm, while EinSumcheck does not require any changes to the verifier. The resulting protocol is also compatible with recent Sumcheck optimizations (e.g., [16]) that reduce proof size.

One-operand cases. For ReduceSum ($ij \rightarrow i$), existing linear-time Sumcheck provers [54] suffice. For Transpose ($ij \rightarrow ji$), the prover simply swaps the corresponding randomness without running Sumcheck. EinSumcheck does have limitations: the compiler does not directly support Trace (e.g., $ii \rightarrow i$), but Trace can be proven by leveraging existing permutation-based techniques [5, 8].

Alg. 1 is the main linear-time prover algorithm for the canonical EinSumcheck relation in Eq. (10). It relies on the proxy procedure FUNCTIONEVALUATIONS (Alg. 2, Appendix G) to precompute the intermediate evaluations used in each Sumcheck round.

G The Proxy Algorithm of Alg. 1

Alg. 2 is the proxy algorithm that computes the intermediate evaluations needed by our main algorithm (Alg. 1). The bookkeeping table

⁸See Alg. 2 in Appendix G.

Algorithm 1 EINSUMCHECKPRODUCT
 $(\tilde{eq}, A^{(eq)}, \tilde{T}^{(1)}, A^{(1)}, \tilde{T}^{(2)}, A^{(2)}, r_1, \dots, r_{\ell+\mu+\eta})$

Require: $\tilde{eq}(r, x)$ on ℓ variables, multilinear $\tilde{T}^{(1)}(x, y)$ on $\ell + \mu$ variables, multilinear $\tilde{T}^{(2)}(x, y, z)$ on $\ell + \mu + \eta$ variables, bookkeeping tables $A^{(eq)} \in \mathbb{F}^{2^\ell}$, $A^{(1)} \in \mathbb{F}^{2^{\ell+\mu}}$, $A^{(2)} \in \mathbb{F}^{2^{\ell+\mu+\eta}}$, randomness $r_1, \dots, r_{\ell+\mu+\eta}$

Ensure: $\ell + \mu + \eta$ Sumcheck messages $\{a_1, \dots, a_{\ell+\mu+\eta}\}$ for

```

1:  $\mathcal{F}^{(eq)} \leftarrow \text{FUNCTIONEVALUATIONS}(\tilde{eq}, A^{(eq)}, r_1, \dots, r_\ell)$ 
2:  $\mathcal{F}^{(1)} \leftarrow \text{FUNCTIONEVALUATIONS}(\tilde{T}^{(1)}, A^{(1)}, r_1, \dots, r_{\ell+\mu})$ 
3:  $\mathcal{F}^{(2)} \leftarrow \text{FUNCTIONEVALUATIONS}(\tilde{T}^{(2)}, A^{(2)}, r_1, \dots, r_{\ell+\mu+\eta})$ 
4: for  $i = 1$  to  $\ell + \mu + \eta$  do
5:   for  $t \in \{0, 1, 2, 3\}$  do ▷ four points for the verifier to
     reconstruct a degree-3 univariate polynomial.
6:    $\hat{r} \leftarrow (r_1, \dots, r_{i-1})$ 
7:    $a_{it} \leftarrow 0$ 
8:   for  $b \in \{0, 1\}^{\ell+\mu+\eta-i}$  do
9:     if  $i > \ell$  then
10:       $c^{(eq)} \leftarrow \tilde{eq}(r_1, \dots, r_\ell)$ 
11:     else
12:       $c^{(eq)} \leftarrow \tilde{eq}(\hat{r}, t, b \bmod 2^{\ell-i})$ 
13:     end if
14:     if  $i > \ell + \mu$  then
15:       $c^{(1)} \leftarrow \tilde{T}^{(1)}(r_1, \dots, r_{\ell+\mu})$ 
16:     else
17:       $c^{(1)} \leftarrow \tilde{T}^{(1)}(\hat{r}, t, b \bmod 2^{\ell+\mu-i})$ 
18:     end if
19:      $a_{it} \leftarrow a_{it} + c^{(eq)} \cdot c^{(1)} \cdot \tilde{T}^{(2)}(\hat{r}, t, b)$ 
20:   ▷ all values are in  $\mathcal{F}^{(\cdot)}$ 
21:   end for
22: end for
23: end for
24: return  $\{a_1, \dots, a_{\ell+\mu+\eta}\}$ 
```

A stores the evaluation vector of a μ -variate multilinear polynomial: initially, $A[b] = \tilde{f}(b)$, where b denotes both an integer and its μ -bit binary representation.

H End-to-End Protocol Specification

Figure 2 specifies the full end-to-end proof generation protocol for zkTransformer, referenced from Sec. 5.2. The high-level description in Sec. 5.2 summarizes the three phases (commit, prove, verify); this appendix gives the full per-phase checklist, including the structured polynomials that the verifier evaluates directly without a commitment.

I Running Example: Query Projection with RoPE in LLaMA2

To assist readers from the AI community in understanding how the building blocks of Sec. 5.3 chain together in a full transformer submodule, we depict a concrete example in Fig. 3. The submodule arises in the self-attention layer of several transformer models (e.g., [50]): it is used when a transformer block converts its input tensor into a query (or key) tensor for self-attention [51] in LLaMA [50].

Protocol: zkTransformer proof generation for an L -layer LLM.

Public: LLM architecture (layer count, dimensions), PCS verification key, structured table polynomials (power-of-2, range), input tokens X (the number is PROMPT_LEN), and output tokens.

Private (prover only): model weights $\{W^{(\ell)}\}$.

Witness generation phase: Upon receiving the input tokens (i.e., input prompt), the prover runs the LLM inference from layer 1 to L and calculates all witnesses including the intermediate tensors and the auxiliary lookup matrices.

Commit phase:

- (1) Commit to model weights $\{W^{(\ell)}\}$ (offline, before receiving input).
- (2) Commit to input tokens X and auxiliary lookup matrices $\{M_i\}$ for range and exponentiation checks based on Shout [41].

Prove phase (backward from output to input via GKR):

- (3) Verifier sends random challenge r for the output layer.
- (4) For each layer $\ell = L, \dots, 1$, building blocks are verified via Sumcheck in reverse order:
 - *Linear operations* (e.g., MatMul, element-wise multiplication, addition): EinSumcheck (Sec. 3) reduces output claims to input claims via Theorem 3.1, without committing to wiring predicates. Addition reduces the output claims into input claims directly at the same challenge points.
 - *Nonlinear operations* (e.g., Softmax, GeLU, SiLU): StructuredExp (Sec. 4) decomposes exponentiation into structured power-of-2 lookup and Taylor polynomial; range checks use Shout with structured range table (Eq. 8).
 - *Shape operations* (e.g., Reshape, Permute): Free reshape via variable reordering; permutation via structured permutation matrix $\tilde{M}$ evaluated directly by verifier.

Each Sumcheck reduces the current claim to evaluation claims on the previous layer's outputs.

- (5) After reaching the input layer, run Shout Sumcheck (Eq. 5) to verify auxiliary lookup matrices $\{M_i\}$.

Verify phase:

- (6) Verify all Sumcheck transcripts.
- (7) Verify PCS openings for model weights $\{W^{(\ell)}\}$, input X , and auxiliary matrices $\{M_i\}$.
- (8) Evaluate structured polynomials directly (no commitment required):
 - Power-of-2 table: $\tilde{T}_{\text{pow}}(y) = \prod_{i=1}^n ((2^{2^{i-1}} - 1)y_i + 1)$
 - Range table: $\tilde{T}_{\text{range}}(y) = \sum_{i=1}^n 2^{i-1} \cdot y_i$
 - Lagrange basis: $\tilde{e}_q(x, y) = \prod_{i=1}^{\mu} (x_i y_i + (1 - x_i)(1 - y_i))$
- (9) If the number of output tokens is larger than 1, check every OUTPUT_TOKENS[i] equals to INPUT_TOKENS[PROMPT_LEN + i].

Figure 2: End-to-end proof generation protocol for zkTransformer. The prove phase walks backward through LLM layers via the GKR protocol, reducing the output claim to input-layer commitments. Unlike prior GKR-based zkML, zkTransformer requires no commitment to wiring predicates (via EinSumcheck) or lookup tables (via StructuredExp).

Algorithm 2 FUNCTIONEVALUATIONS($\tilde{f}, A, r_1, \dots, r_\mu$) (adapted from [54])

Require: Multilinear function $\tilde{f}$ on μ variables, bookkeeping table $A \in \mathbb{F}^{2^\mu}$, randomness $r_1, \dots, r_\mu$, the needed number of evaluation m

Ensure: All values $\tilde{f}(r_1, \dots, r_{i-1}, t, b_{i+1}, \dots, b_\mu)$ for $i \in [\mu]$, $t \in \{0, 1, 2, 3\}$, $b \in \{0, 1\}^{\mu-i}$

```

1:  $\mathcal{F} \leftarrow \emptyset$ 
2: for  $i = 1$  to  $\mu$  do
3:   for all  $b \in \{0, 1\}^{\mu-i}$  do  $\triangleright b$  is both a number and its binary representation
4:     for  $t \in \{0, 1, 2, 3\}$  do
5:        $\tilde{f}(r_1, \dots, r_{i-1}, t, b) \leftarrow A[b] \cdot (1 - t) + A[b + 2^{\mu-i}] \cdot t$ 
6:        $\mathcal{F} \leftarrow \mathcal{F} \cup \{\tilde{f}(r_1, \dots, r_{i-1}, t, b)\}$ 
7:     end for
8:      $A[b] \leftarrow A[b] \cdot (1 - r_i) + A[b + 2^{\mu-i}] \cdot r_i$ 
9:   end for
10: end for
11: return  $\mathcal{F}$ 

```

MatMul updates the embedding, Reshape splits it into multiple heads [51], and RoPE encodes the position information into the embedding for each head [44]. As suggested by its original authors [44], the RoPE embedding for a d -dimensional tensor t can be computed by:

$$\begin{pmatrix} t_1 \\ t_2 \\ t_3 \\ t_4 \\ \vdots \\ t_{d-1} \\ t_d \end{pmatrix} \odot \begin{pmatrix} \cos(m\theta_1) \\ \cos(m\theta_1) \\ \cos(m\theta_2) \\ \cos(m\theta_2) \\ \vdots \\ \cos(m\theta_{d/2}) \\ \cos(m\theta_{d/2}) \end{pmatrix} + \begin{pmatrix} t_2 \\ t_1 \\ t_4 \\ t_3 \\ \vdots \\ t_d \\ t_{d-1} \end{pmatrix} \odot \begin{pmatrix} -\sin(m\theta_1) \\ \sin(m\theta_1) \\ -\sin(m\theta_2) \\ \sin(m\theta_2) \\ \vdots \\ -\sin(m\theta_{d/2}) \\ \sin(m\theta_{d/2}) \end{pmatrix}$$

given that m is the position of t in a sequence for LLM inference, $\odot$ denotes element-wise multiplication, and $\theta_i = 10000^{-2(i-1)/d}$, $i \in \{1, 2, \dots, d/2\}$. The sin and cos tensors can be precomputed and are denoted as s and c in Fig. 3.

We also show the corresponding Rust implementation (Listing 1) to demonstrate that zkTransformer supports a user-friendly interface for AI practitioners to build GKR circuits for LLM inference, in a similar way as they build standard LLM models in PyTorch [33] or Tensorflow [1].

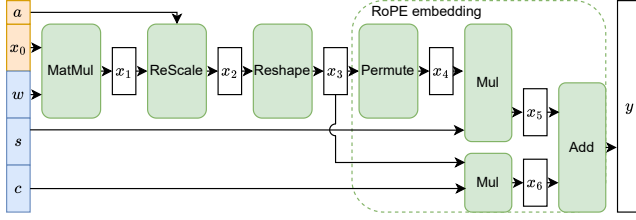


Figure 3: Example of a GKR circuit in zkTransformer for a submodule in the self-attention layer. Tensors are represented by rectangles with right-angle corners, and operations are represented by green rounded rectangles. The prover must commit to the colored tensors: blue tensors can be committed to ahead of proving, while the commitment time for orange tensors is included in the prover time. The arrows indicate the data flow during witness generation (i.e., LLM inference).

Listing 1 Corresponding Rust code in zkTransformer for the GKR circuit in Fig. 3.

```
// llm builds a GKR circuit from high-level tensor
↪ operators
// Shapes of x_0 and w: [B, S, H] & [H, H]
// H = head_num * head_dim
let rescale = true;
let x_2 = llm.einsum("bsi,ij->bsj".to_string(), vec![x_0,
↪ w], rescale)[0];
let x_3 = llm.reshape(x_2, vec![batch, seq_len, head_num,
↪ head_dim])[0];
let y = llm.ropc(x_3, 0)[0];
```

While witness generation follows the direction of arrows in Fig. 3 during LLM inference, proof generation walks backward from y to x_0 (see Sec. 2.4.2). Accordingly, we explain how the building blocks of Sec. 5.3 are invoked to prove each operation in Fig. 3 in reverse order. To prove the correctness of the output (i.e., y), a prover $\mathcal{P}$ first sends polynomial commitments of all inputs (i.e., the colored tensors). The verifier $\mathcal{V}$ then sends a challenge r and asks $\mathcal{P}$ to show $\tilde{y}(r)$.

Step 1: Add (root of the circuit). Following the *Element-wise addition* block, $\mathcal{P}$ reduces

$$\tilde{y}(r) = \tilde{x}_5(r) + \tilde{x}_6(r)$$

by Schwartz–Zippel [40, 57]. No Sumcheck is required; the two claims $\tilde{x}_5(r)$ and $\tilde{x}_6(r)$ are forwarded upstream.

Step 2: Element-wise Mul on both branches. The tensor $x_6 = x_3 \odot c$ is handled by the *Einsum-expressible linear operations* block. EinSumcheck arithmetizes the element-wise multiplication (i.e., $i, i \rightarrow i$) as

$$\tilde{x}_6(r) = \sum_z \tilde{\text{eq}}(r, z) \tilde{x}_3(z) \tilde{c}(z).$$

The Sumcheck produces a challenge r' and yields claims $\tilde{x}_3(r')$ and $\tilde{c}(r')$; $\tilde{\text{eq}}(r, r')$ is evaluated directly by $\mathcal{V}$ (Lemma 3.3), and $\tilde{c}(r')$ is discharged by a PCS opening on the committed tensor c . Similarly, $\mathcal{P}$ reduces $\tilde{x}_5(r)$ to a claim $\tilde{x}_4(r'')$ at some challenge r'' .

(We skip ReScale operations between Mul and Add in the figure to ease presentation; they are inserted in the real proof as described below.)

Step 3: General permutation (RoPE neighbor swap). The relation $x_4 = M \cdot x_3$ is handled by the *General permutation* block, where M is the neighbor-swap permutation matrix

$$M = \begin{pmatrix} 0 & 1 & 0 & 0 & \cdots & 0 & 0 \\ 1 & 0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 0 & 1 & \cdots & 0 & 0 \\ 0 & 0 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & 0 & 0 & \cdots & 1 & 0 \end{pmatrix}.$$

With EinSumcheck, $\mathcal{P}$ reduces $\tilde{x}_4(r'')$ to evaluation claims for $\tilde{M}$ and $\tilde{x}_3$ via $\tilde{x}_4(r'') = \sum_z \tilde{M}(r'', z) \tilde{x}_3(z)$. As shown in Sec. 5.3, $\tilde{M}(w, z) = \tilde{\text{eq}}(w_1, 1 - z_1) \prod_{i=2}^{\log d} \tilde{\text{eq}}(w_i, z_i)$, which $\mathcal{V}$ evaluates directly [8].

Step 4: Merging the two claims on $\tilde{x}_3$. At this point, $\mathcal{P}$ must prove two claims on $\tilde{x}_3$ —one from the Mul branch (Step 2) and one from the Permute branch (Step 3). Following the *Merging multiple claims* subroutine [48, Sec. 4.5.2], the two claims are reduced to a single claim $\tilde{x}_3(\gamma)$ at a fresh challenge γ .

Step 5: Reshape. Since the reshape from x_2 to x_3 only regroups dimensions (head-splitting), the *Reshape* block reduces $\tilde{x}_3(\gamma)$ to $\tilde{x}_2(\gamma')$ where γ' reorders the random field elements in γ . No Sumcheck is required.

Step 6: ReScale. The ReScale block between the MatMul output x_1 and its rescaled counterpart x_2 enforces $x_1 \approx 2^s \cdot x_2$, where $2^{-s} = S$ is the scale factor (Sec. 2.3). Following the *ReScale* block, $\mathcal{P}$ shows

$$\tilde{x}_1(\gamma') = 2^s \cdot \tilde{x}_2(\gamma') + \tilde{\text{res}}(\gamma'),$$

and proves $\tilde{\text{res}}$ lies in a small range via the Shout Sumcheck

$$\tilde{\text{res}}(\gamma') = \sum_{z \in \{0,1\}^s} \tilde{a}(\gamma', z) \tilde{T}(z),$$

where $\tilde{T}$ is the structured range-table MLE. This is why $\mathcal{P}$ commits to the auxiliary sparse Boolean matrix a (one of the orange tensors in Fig. 3) at the start of the GKR protocol. At the end of the protocol, $\mathcal{V}$ evaluates the structured $\tilde{T}$ directly and $\mathcal{P}$ provides a PCS opening for $\tilde{a}$.

Step 7: MatMul (input of the submodule). The final operation is the MatMul that produces x_1 from x_0 and w . Using the *Einsum-expressible linear operations* block, $\mathcal{P}$ and $\mathcal{V}$ reduce $\tilde{x}_1(\gamma')$ to evaluation claims on $\tilde{x}_0$ and $\tilde{w}$, which $\mathcal{P}$ discharges by PCS openings on the committed tensors. In the end, $\mathcal{V}$ accepts the GKR proof if all Sumcheck proofs and PCS openings are valid.

J Proof-size breakdown

Table 8 reports the breakdown of the 3.88 MB proof at total sequence length 32. The opening proofs account for 63.7% of the proof size, which is consistent with the $O(N^{1/3})$ verifier-side opening cost of KZH-3 dominating over the $O(\log N)$ Sumcheck transcripts (for both EinSumcheck proofs and Shout lookup proofs). Therefore,

Table 8: Total Proof-size Breakdown at Sequence Length 32 on GPT-2.

Component	Size	%
Opening proofs	2529.28 KB	63.7
Sumcheck proofs	1445.34 KB	36.3
Total	3973.12 KB	100.0

although KZH-3 provides practical prover performance for polynomial openings, it is possible to replace it with other schemes such as Dory [27] to greatly reduce the proof size of zkTransformer.